

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA ĐIỆN – ĐIỆN TỬ



ĐỒ ÁN 1 - THIẾT KẾ VI MẠCH

**NGHIÊN CỨU VÀ TRIỂN KHAI KỸ THUẬT BEAMFORMING
SỬ DỤNG MẠNG NEURON TRÊN FPGA**

SVTH: Huỳnh Trung Kiên

MSSV:2311729

GVHD: TS Hoàng Mạnh Hà

Thành phố Hồ Chí Minh, ngày 30 tháng 05 năm 2026

KHOA ĐIỆN – ĐIỆN TỬ		NHIỆM VỤ ĐỒ ÁN	
		<i>(Chú ý: sinh viên phải dán tờ này vào trang thứ nhất của bản thuyết minh)</i>	
HỌ VÀ TÊN:	Huỳnh Trung Kiên	MSSV:	2311729
NGÀNH:	Thiết kế Vi mạch	LỚP:	DD23VIM1
1. Đầu đề đồ án: Nghiên cứu và triển khai kỹ thuật Beamforming sử dụng mạng neuron trên FPGA			
2. Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):			
-	Nghiên cứu tài liệu		
-	Lấy số liệu mô phỏng		
-	Thiết kế phân cứng bằng SystemVerilog		
-	Huấn luyện, tạo các trọng số cho mô hình mạng neuron		
-	Nhúng các trọng số vào thiết kế trên FPGA và chạy thử nghiệm		
3. Ngày giao nhiệm vụ đồ án:			
4. Ngày hoàn thành nhiệm vụ:			
5. Họ và tên người hướng dẫn:			Phần hướng dẫn:
-	Hoàng Mạnh Hà	Khoa Điện – Điện tử	100 %
Nội dung và yêu cầu Đồ án đã được thông qua Chủ trì ngành.			

Ngàytháng năm

CHỦ TRÌ NGÀNH
(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN
(Ký và ghi rõ họ tên)

PHẦN DÀNH CHO KHOA

Người duyệt (chấm sơ bộ):

Đơn vị:

Ngày bảo vệ:

Điểm tổng kết:

Nơi lưu trữ:

MỤC LỤC

Lời nói đầu	1
Chương 1: Giới thiệu	3
Chương 2: Nền tảng lý thuyết về Tạo búp sóng thích nghi	5
2.1 Giới thiệu	5
2.2 Cơ sở về Mảng ăng-ten và Tạo búp sóng	5
2.2.1 Kiến trúc Tạo búp sóng số	8
2.2.2 Tạo búp sóng tất định	10
2.2.3 Hiệu ứng mảng trong tạo búp sóng	13
2.3 Tạo búp sóng thích nghi	17
2.3.1 Tạo búp sóng thích nghi MVDR	19
Chương 3: Học máy ứng dụng vào tạo búp sóng thích nghi	27
4.1 Nền tảng về Học sâu	27
4.1.1 Kiến trúc CNN	27
4.1.2 Phát triển mô hình CNN trong TensorFlow	29
4.2 Triển khai CNN trên FPGA	31
Chương 5: Các kết quả đạt được & hướng phát triển	33
Kết quả huấn luyện bộ trọng số:	33
5.2 Kết luận và hướng phát triển:	34
Phụ lục A: Mã nguồn phần mềm	35
Phụ lục B: Mã nguồn thiết kế SystemVerilog	44
TÀI LIỆU THAM KHẢO	45

Lời nói đầu

Trong các hệ thống Đa đầu vào – Đa đầu ra (MIMO) hiện đại như mạng di động và Wi-Fi, một mảng gồm các phần tử anten được sử dụng để điều hướng tín hiệu vô tuyến (RF) trong không gian. Quá trình này nhằm mục đích thay đổi giản đồ bức xạ của anten để đạt được tỷ số tín hiệu trên nhiễu và tạp âm (SINR) cao hơn. Kỹ thuật tạo búp sóng số (DBF) thực hiện hiệu ứng điều hướng này bằng cách áp dụng các hệ số trọng số cho các phần tử anten – tương tự như cơ chế của một bộ lọc số – giúp điều chỉnh pha và biên độ của các tín hiệu thu hoặc phát.

Tuy nhiên, các hệ thống mảng có số lượng phần tử lớn với bậc tự do (DOF) cao thường đối mặt với nhiều thách thức khi triển khai các thuật toán thích nghi này trong thực tế. Các phương pháp tính toán ma trận thống kê này đòi hỏi khối lượng tính toán khổng lồ, hoặc buộc phải chấp nhận các giải pháp giản lược không tối ưu để kịp thời cung cấp các trọng số thích nghi cho ứng dụng, đặc biệt là dưới giới hạn về năng lực xử lý của hệ thống. Ví dụ, các thiết bị truyền thông MIMO bị ràng buộc nghiêm ngặt về kích thước, trọng lượng và công suất tiêu thụ (tiêu chuẩn SWaP) thường bị giới hạn khả năng tính toán do chỉ sử dụng các bộ xử lý công suất thấp hoặc chip FPGA quy mô nhỏ.

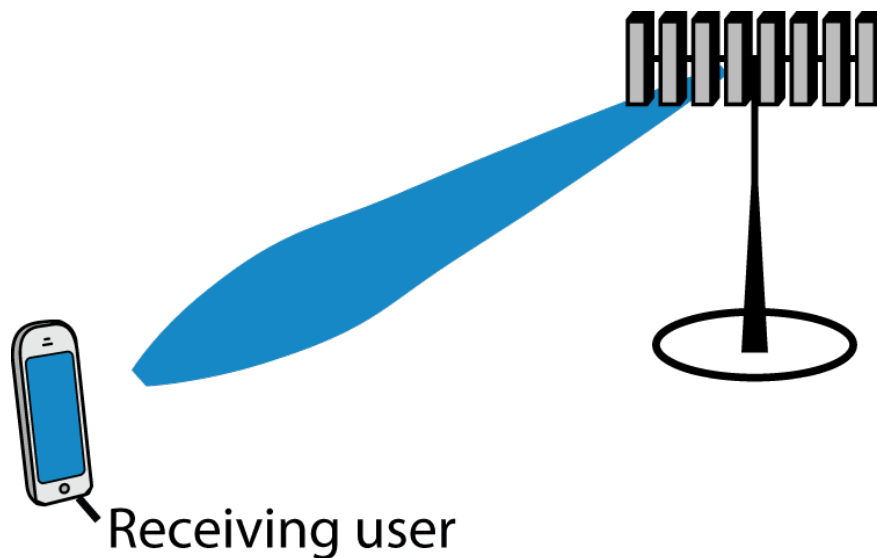
Do đó, nghiên cứu trong luận văn này trình bày những tiến bộ mới nhằm giải quyết các thách thức của hệ thống MIMO thích nghi theo hai hướng tiếp cận. Thứ nhất, nghiên cứu chứng minh rằng các bước phát triển của Học máy (ML) và Mạng nơ-ron sâu (DNN) có thể được áp dụng trực tiếp để giải quyết bài toán ước lượng trọng số tạo búp sóng thích nghi tối ưu vốn có độ phức tạp tính toán rất cao, thông qua một cấu trúc Mạng nơ-ron tích chập (CNN) được thiết kế riêng. Thứ hai, mô hình CNN tạo búp sóng thích nghi sau khi phát triển được chứng minh là có khả năng ánh xạ hiệu quả vào tài nguyên logic khả trình của FPGA để cập nhật các hệ số thích nghi theo thời gian thực.

Giải pháp triển khai học máy này được so sánh với kiến trúc FPGA tiên tiến nhất hiện nay sử dụng thuật toán Bình phương tối thiểu đệ quy (RLS) truyền thống. Kết quả thực nghiệm chứng minh rằng kiến trúc đề xuất cung cấp các trọng số tạo búp sóng thích nghi nhanh hơn, đồng thời tiêu tốn ít tài nguyên logic FPGA hơn. Sự cắt giảm đáng kể về độ trễ xử lý và không gian cấu trúc FPGA cho phép các bộ xử lý MIMO bị ràng buộc SWaP triển khai kỹ thuật định hướng búp sóng thích nghi cho các hệ thống có số lượng kênh lớn hơn — điều vượt quá khả năng đáp ứng của các phương pháp tính toán toán học truyền thống hiện tại.

Trong phạm vi của nghiên cứu này, đề tài tập trung triển khai mạng nơ-ron chuyên dụng, đồng thời đưa ra giải pháp tối ưu hóa cấu hình và thực thi mạch số trên nền tảng phần cứng. Để đạt được những kết quả thực nghiệm trên, em xin bày tỏ lòng biết ơn chân thành và sâu sắc nhất đến các thầy cô, đặc biệt là thầy **Hoàng Mạnh Hà** đã nhiệt tình định hướng, dẫn dắt và tạo điều kiện cho em đạt được thành quả ngày hôm nay.

Chương 1: Giới thiệu

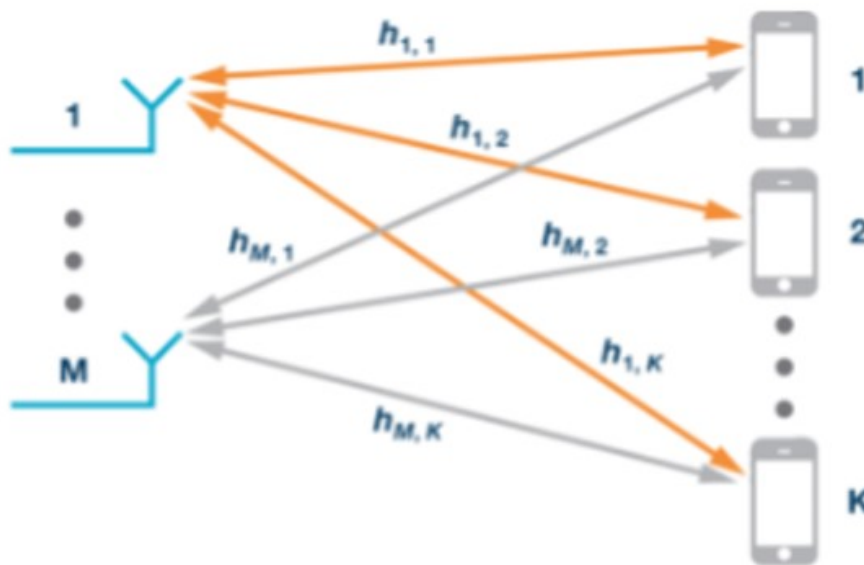
Các hệ thống Đa đầu vào – Đa đầu ra (MIMO) hiện đại, chẳng hạn như những hệ thống được sử dụng trong công nghệ truyền thông di động và Wi-Fi, thường được phát triển để phục vụ tối ưu nhiều người dùng cuối. Để làm điều này, nhiều phần tử ăng-ten thường được phối hợp theo cách cho phép đồ thị bức xạ được "điều khiển" trong không gian theo hướng của một người dùng cụ thể, để người dùng đó nhận được cường độ tín hiệu cao, trong khi những người dùng khác bị suy hao để không gây nhiễu kênh truyền thông của người dùng đó. Quá trình tạo bức sóng này có thể được hình dung như một bức sóng định hướng như trong Hình 1.1.



[Hình 1.1: Tạo bức sóng tín hiệu từ hệ thống MIMO đến người dùng]

Tuy nhiên, do các hệ thống MIMO thực tế thường hoạt động trong môi trường động với các nguồn nhiễu liên tục thay đổi, các kỹ thuật Tạo bức sóng thích nghi đã được sử dụng để triệt tiêu các nguồn nhiễu khỏi kênh truyền thông mong muốn. Tuy nhiên, các hệ thống mảng có số lượng phần tử lớn, với bậc tự do (DOF) cao, đòi hỏi xử lý đáng kể trong tính toán và áp dụng thực tế các thuật toán tạo bức sóng thích nghi. Đối với các triển khai nhúng bị ràng buộc SWaP, các yêu cầu xử lý này có thể quá lớn để một hệ thống tính toán trọng số tạo bức sóng thích nghi kịp thời.

Vấn đề càng trở nên nghiêm trọng hơn trong các hệ thống MIMO quy mô lớn, nơi nhiều người dùng trong cùng hoặc dải tần lân cận cạnh tranh băng thông và gây nhiễu lẫn nhau. Do đó, các thuật toán tạo bức sóng thích nghi hiệu quả sẽ cần thiết để hỗ trợ thực tế các hệ thống MIMO thế hệ tiếp theo.



[Hình 1.2: Nhiều người dùng đồng thời trong hệ thống MIMO]

Do độ phức tạp xử lý của tạo búp sóng thích nghi tăng theo hàm mũ với số kênh, một thuật toán tạo búp sóng thích nghi hiệu quả hơn so với các phương pháp truyền thống có thể cho phép một thiết bị biên – chẳng hạn như trạm phát sóng 5G – tính toán trực tiếp trọng số tạo búp sóng thích nghi tại biên. Để hỗ trợ mục tiêu này, nghiên cứu này sẽ xem xét nền tảng phương pháp tạo búp sóng thích nghi truyền thống và khả năng ứng dụng trên FPGA công suất thấp, như thường được sử dụng trong các thiết bị truyền thông MIMO hiện đại, chẳng hạn như đầu vô tuyến 5G.

Sau khi thiết lập triển khai cơ sở sử dụng kiến trúc FPGA tiên tiến nhất hiện nay, nghiên cứu này sẽ trình bày một phương pháp mới trong việc áp dụng những tiến bộ gần đây trong Học sâu vào bộ bài toán tính toán trọng số Tạo búp sóng thích nghi. Để trình bày một triển khai có thể triển khai thực tế, công trình này cũng sẽ trình bày một kiến trúc được triển khai trên cấu trúc logic khả lập trình của FPGA để tính toán trọng số thích nghi theo cách hiệu quả hơn so với các triển khai trước đây.

Chương 2: Nền tảng lý thuyết về Tạo búp sóng thích nghi

2.1 Giới thiệu

Trong phần này, khái niệm và ứng dụng của tạo búp sóng, cũng như phương pháp tạo búp sóng thích nghi, được giới thiệu. Người đọc được giả định đã quen thuộc với các nguyên lý cơ bản của kỹ thuật xử lý tín hiệu rời rạc, chẳng hạn như lọc Đáp ứng xung hữu hạn (FIR).

2.2 Cơ sở về Mảng ăng-ten và Tạo búp sóng

Lọc là một phép toán thường dùng trong xử lý tín hiệu; trong miền rời rạc, các mẫu được đưa qua một tập hợp các hệ số bộ lọc để thực hiện phép tích chập và đạt được đáp ứng mong muốn. Tương tự như lọc theo thời gian, một mảng cảm biến có thể được lọc theo không gian để tạo ra đáp ứng mong muốn trên các phần tử.

Cụ thể trong bối cảnh các mảng ăng-ten tần số vô tuyến (RF), lọc không gian này có thể được sử dụng để tối ưu hóa đồ thị tổng thể của ăng-ten, trong một quá trình thường được gọi là tạo búp sóng. Tối ưu hóa không gian cụ thể thường phụ thuộc vào ứng dụng, tuy nhiên tạo búp sóng thường được coi là phương pháp điều khiển búp sóng, nơi độ lợi được cung cấp theo một hướng cụ thể mong muốn – tương đối so với phía trước của mảng – với suy hao ở các góc khác. Tạo búp sóng có thể được thực hiện trên cả chức năng phát và thu của bất kỳ mảng nào, còn được gọi là tính tương hỗ của mảng. Tạo búp sóng còn bao gồm các mảng và ứng dụng phi RF, chẳng hạn như các bộ chuyển đổi âm thanh được sử dụng trong mảng SONAR.

Hoạt động cơ bản của tạo búp sóng được rút ra từ các tính chất giao thoa xây dựng và phá vỡ của các sóng truyền lan trong các hệ thống mảng pha. Các phần tử mảng riêng lẻ dịch pha của tín hiệu thu hoặc phát để tạo ra đồ thị mảng ở trường xa mong muốn.

Quá trình dịch pha này có thể được thực hiện bằng phương tiện kỹ thuật số hoặc tương tự. Ví dụ trong Hình 2.1, các khối Δ có thể là các đơn vị dịch pha tương tự thực hiện điều khiển hướng búp sóng trong miền RF. Trong trường hợp này, mỗi đơn vị dịch pha được gắn với một phần tử ăng-ten riêng lẻ và được kết nối đến một bộ thu duy nhất – chẳng hạn như ADC – và/hoặc một bộ phát duy nhất – chẳng hạn như DAC. Ưu điểm của hệ thống như vậy là sự đơn giản trong các điện tử số và RF, vì chỉ có một ADC và/hoặc DAC (và có thể một hệ thống trộn/heterodyne cho mảng), tuy nhiên hệ thống kém linh hoạt hơn ở chỗ chỉ có thể điều khiển theo một hướng tại một thời điểm. Đối với các hệ thống Đơn đầu vào – Đơn đầu ra (SISO), kiến trúc này có thể đủ dùng.

Tuy nhiên đối với các hệ thống Đa đầu vào – Đa đầu ra (MIMO) hoặc các hệ thống khác cần tính linh hoạt cao hơn, các khối dịch pha Δ này cũng có thể được thực hiện trong miền kỹ thuật số. Trong trường hợp này, mỗi phần tử ăng-ten có thể được coi là kết nối trực tiếp với ADC và/hoặc DAC và các dịch pha liên quan có thể được thực hiện trong logic kỹ thuật số – chẳng hạn như trong FPGA kết nối trực tiếp với từng ADC/DAC – và sau đó được cộng kết hợp để tạo thành búp sóng mong muốn. Nhược điểm của hệ thống tạo búp sóng số là tăng độ phức tạp và chi phí do mỗi kênh yêu cầu điện tử RF và lấy mẫu phải đồng bộ pha, tuy nhiên ưu điểm là hệ thống linh hoạt hơn nhiều trong cách áp dụng dịch pha, đồng thời tạo cơ hội để hệ thống tạo nhiều búp sóng không gian cùng một lúc.

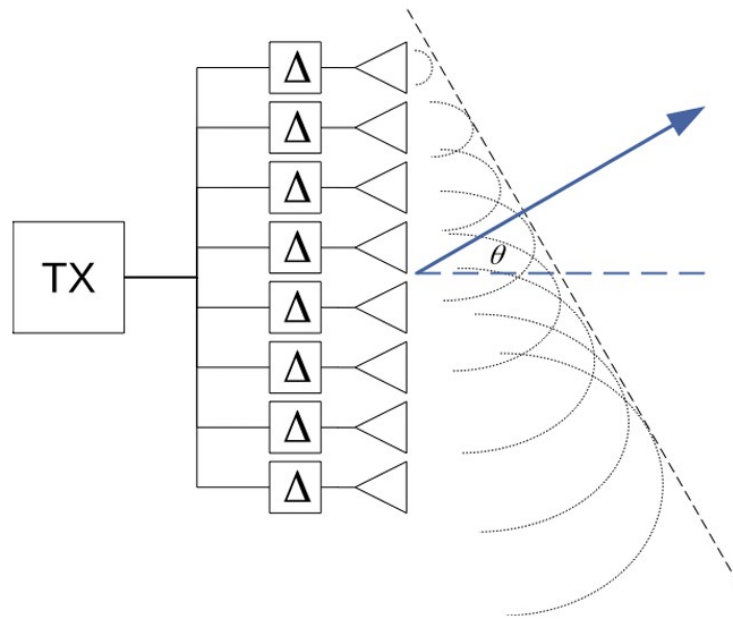
Đối với các mảng truyền thông MIMO, các tính chất độ lợi định hướng và suy hao này có thể được khai thác để phục vụ nhiều người dùng, chẳng hạn như trong Ghép kênh không gian (Spatial Multiplexing) nơi các người dùng riêng biệt được gán định ở các vị trí hoặc hướng không gian khác nhau, vì vậy tạo búp sóng số với nhiều búp sóng có thể được sử dụng để nhắm đến từng người dùng một cách độc lập cùng một lúc.

Mảng tuyến tính đều (ULA) được định nghĩa là một mảng gồm N phần tử cách nhau đều một khoảng cách d dọc theo trục tuyến tính. Mỗi kênh RF được lấy mẫu đồng bộ để các mẫu kỹ thuật số được căn chỉnh theo thời gian trên tất cả các kênh nhằm thực hiện xử lý kết hợp. Kiến trúc tạo búp sóng số chuẩn cho trường hợp ULA có thể thấy trong Hình 2.2.

[Hình 2.2: Bộ tạo búp sóng ULA]

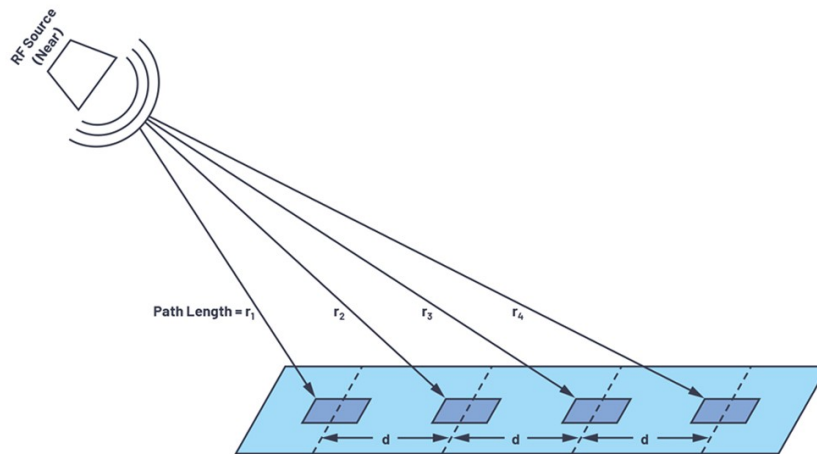
Có thể thấy rằng khi xử lý tín hiệu từ trường xa tác động lên mảng với góc θ_0 , sự chênh lệch đường truyền lan L giữa các phần tử trong ULA được cho bởi Phương trình 2.1:

$$L(n) = nd \cdot \sin(\theta_0), \quad 0 \leq n \leq N-1 \quad (2.1)$$



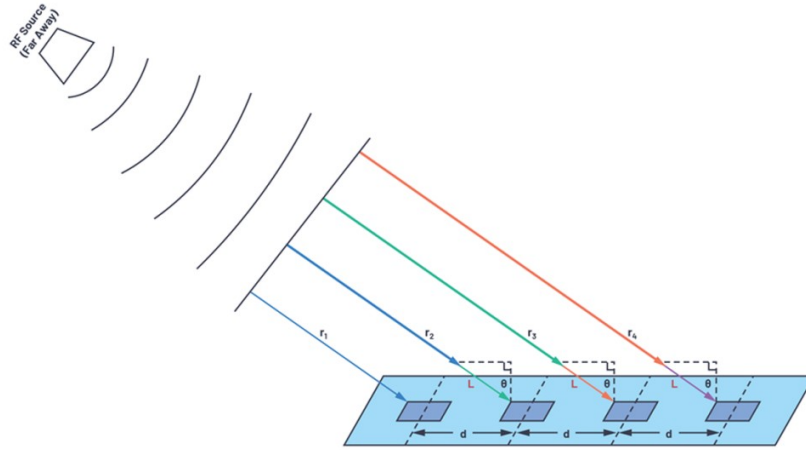
[Hình 2.1: Điều khiển mặt sóng mảng pha]

Lý do chúng ta giả định đặc tính trường xa cho phần lớn công trình này là để đơn giản hóa toán học và các phép toán của mảng pha. Trong trường gần, một nguồn phát ở rất gần mảng đến mức góc tới của năng lượng thu được là khác nhau cho mỗi phần tử do mặt sóng cầu của nguồn.



[Hình: Đáp ứng trường gần]

Tuy nhiên trong trường xa, các mặt sóng trở nên gần như tuyến tính, và mỗi phần tử thu thấy góc tới tương đương θ của sóng đến. Điểm cụ thể mà một hệ thống nhất định đang hoạt động trong trường xa phụ thuộc vào nhiều yếu tố, tuy nhiên có thể tìm thấy phương trình tổng quát từ Phương trình 2.2 dựa trên đường kính ăng-ten D và bước sóng λ :



[Hình: Đáp ứng trường xa]

$$\text{Trường xa} > 2D^2 / \lambda \quad (2.2)$$

2.2.1 Kiến trúc Tạo búp sóng số

Sau khi vào miền kỹ thuật số, có nhiều cách để thực hiện dịch pha trên các tín hiệu băng cơ đã lấy mẫu, chủ yếu thông qua dịch trễ thời gian hoặc nhân với phasor phức. Quyết định lấy cách tiếp cận nào chủ yếu được quyết định bởi các đặc tính hệ thống RF, tuy nhiên đối với các tín hiệu băng hẹp được sử dụng trong tài liệu này, dịch pha thường tiết kiệm hơn so với dịch thời gian.

Dịch trễ thời gian thực sự hoạt động bằng cách bù trừ sự chênh lệch thời gian của một mặt sóng tác động lên từng phần tử khác nhau sao cho khi các kênh đã được bù trễ được cộng kết hợp lại, tín hiệu mong muốn được tăng cường độ lợi xử lý, trong khi các tín hiệu từ các hướng khác bị suy hao. Cụ thể, mỗi phần tử ăng-ten sẽ thấy một dịch thời gian của cùng một tín hiệu dựa trên chiều dài đường truyền cụ thể L và tốc độ truyền lan gần bằng tốc độ ánh sáng c :

$$t_{\text{Delay}(n)} = L(n)/c, \quad 0 \leq n \leq N-1 \quad (2.3)$$

Tuy nhiên, đạt được trễ thời gian chính xác yêu cầu cho một hướng điều khiển nhất định có thể không thực tế. Ví dụ, trong logic kỹ thuật số, bước lượng tử hóa trễ thời gian bị giới hạn bởi tần số xung nhịp: với vùng xung nhịp 200 MHz trong FPGA, mỗi tín hiệu chỉ có thể bị trễ theo bội số nguyên của bước 5 nanosecond.

Do đó, thay vì thực hiện trễ thời gian thực sự, thường có thể xấp xỉ chính xác dịch thời gian bằng dịch pha được áp dụng, đặc biệt đối với các tín hiệu băng hẹp. Khi L là một phần của bước sóng λ , dịch pha tương đương ϕ của tín hiệu tác động tại góc tới θ có thể được rút ra từ Phương trình 2.4:

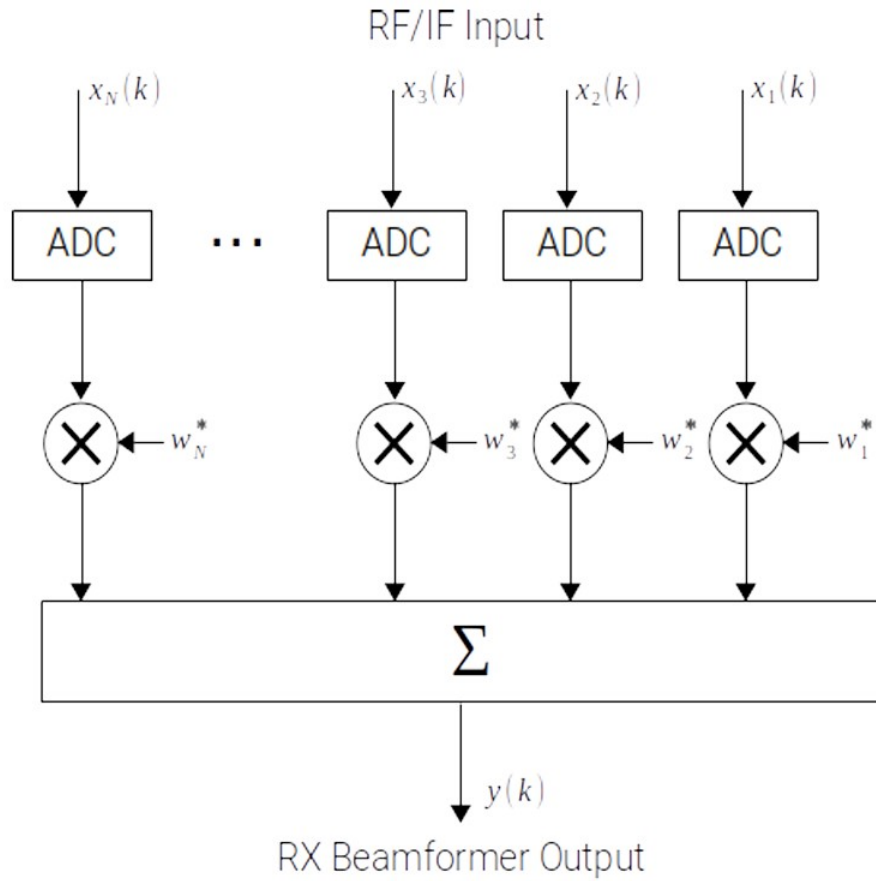
$$\phi = (2\pi d/\lambda) \cdot \sin(\theta), \quad -\pi/2 \leq \theta \leq \pi/2 \quad (2.4)$$

Ràng buộc trên θ trong Phương trình 2.4 để ϕ hợp lệ là $d < \lambda/2$, sao cho không có sự mơ hồ giữa giá trị góc tới và dịch pha mong muốn. Yêu cầu khoảng cách nửa bước sóng này có thể được coi trực tiếp là tương tự không gian của định lý lấy mẫu Nyquist, trong đó tương tự quy định $B < f_s/2$.

Do đó, trong các hệ thống băng hẹp thiết kế cho một tần số sóng mang cụ thể, khoảng cách phần tử mảng thường bằng nửa bước sóng tín hiệu, dẫn đến đơn giản hóa thêm của dịch pha từng phần tử:

$$\phi = \pi \cdot \sin(\theta), \text{ khi } d = \lambda/2 \quad (2.5)$$

Khi một mảng sử dụng dịch pha số để thực hiện tạo búp sóng khi thu, kiến trúc cơ bản có thể thấy trong Hình 2.5, trong đó mỗi kênh ADC được dịch pha bởi một giá trị trọng số phức w_n . Dữ liệu mẫu số hóa từ mỗi kênh $x_n(k)$ được giả định là phức – còn được gọi là dữ liệu Đồng pha và Vuông pha (I/Q) – để giữ lại thông tin pha từ mỗi kênh, trong đó k là chỉ số thời gian.



[Hình 2.5: Tạo búp sóng số ULA khi thu]

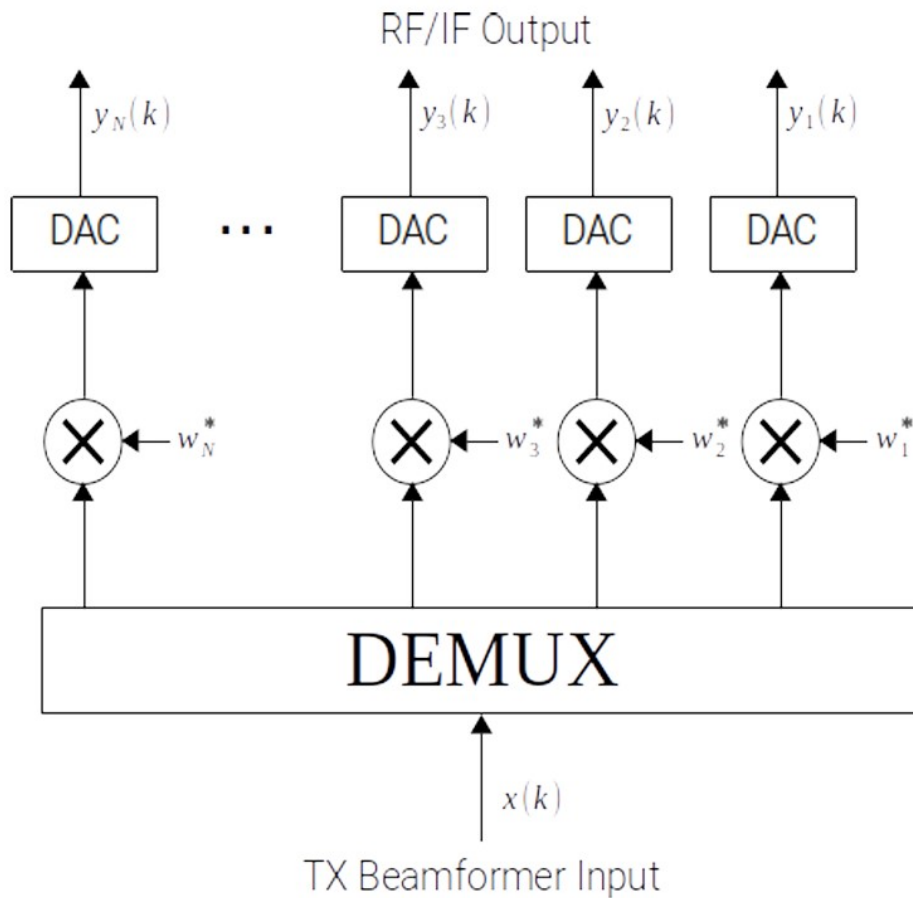
Tín hiệu đầu ra tạo búp sóng $y(k)$ được tạo thành bằng cách cộng các tích của liên hợp phức của trọng số w_n và các tín hiệu đầu vào trên mỗi kênh $x_n(k)$, như trong Phương trình 2.6:

$$y(k) = \sum_{i=0 \text{ đến } N-1} w_i^* \cdot x_i(k) \quad (2.6)$$

Tương đương trong ký hiệu vector, đây là tích chấm của vector cột trọng số phức $N \times 1$ $w = [w_1, w_2, \dots, w_N]^T$ và vector cột mẫu phức $N \times 1$ $x(k)$ tại thời điểm quan sát k :

$$y(k) = w^H \cdot x(k) \quad (2.7)$$

Lưu ý rằng ký hiệu mũ $(\cdot)^H$ biểu diễn phép chuyển vị Hermitian (liên hợp phức) áp dụng cho vector trọng số. Ngược lại trong trường hợp phát, luồng dữ liệu ngược chiều: một tín hiệu phát đơn được tách ra (DEMUX) tới từng kênh phát riêng lẻ, nơi trọng số phức được nhân, và đầu ra của mảng kết quả có búp sóng được điều khiển theo hướng nhất định.



[Hình 2.6: Tạo búp sóng số ULA khi phát]

2.2.2 Tạo búp sóng tắt định

Một bộ tạo búp sóng chuẩn, với các trọng số tạo búp sóng quiescent w_n , có thể được gọi là bộ tạo búp sóng tắt định. Hệ thống được coi là quiescent theo nghĩa là việc tính toán các trọng số tạo búp sóng chỉ cần phụ thuộc vào hướng điều khiển mong muốn của mảng, với tất cả các đặc tính hệ thống khác là tĩnh hoặc không được đưa vào quá trình tính toán.

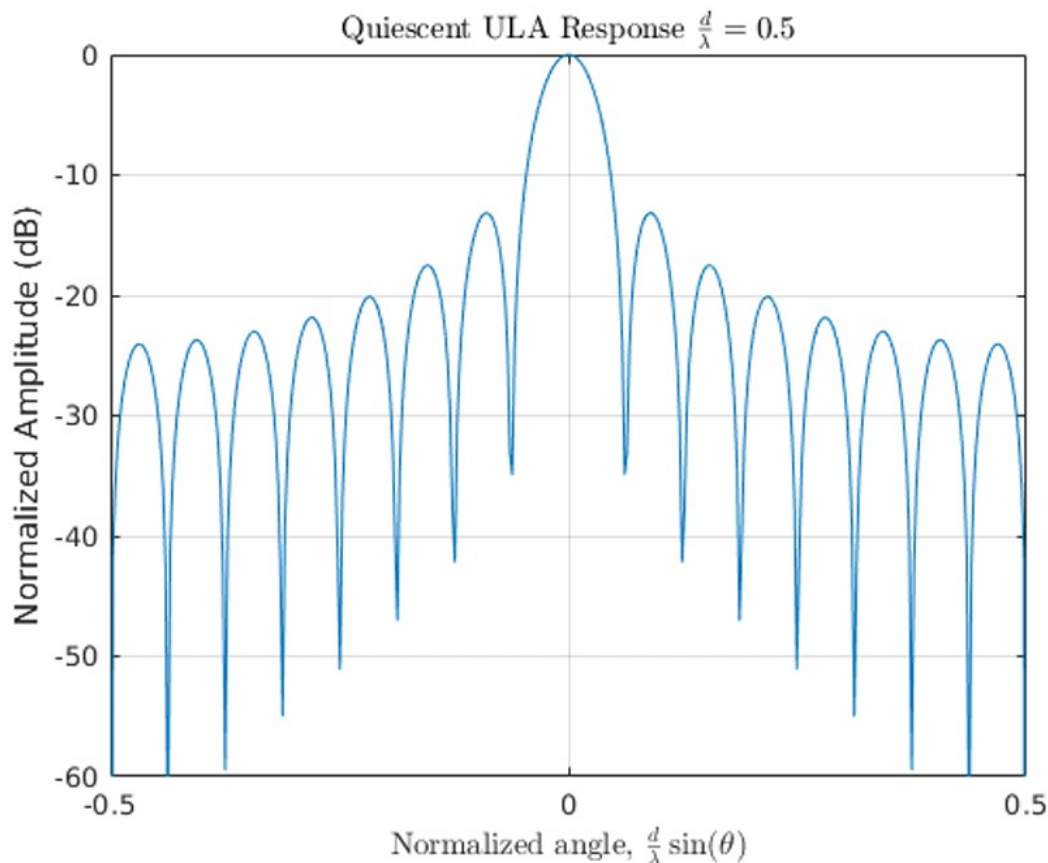
Đối với tín hiệu băng hẹp, vector đáp ứng không gian phức s_n được hình thành từ đường bao phasor băng cơ tại mỗi phần tử ULA:

$$s_n = e^{j2\pi(n-1) \cdot d/\lambda \cdot \sin(\theta_0)}, \quad 0 \leq n \leq N-1 \quad (2.8)$$

Phương trình 2.8 là hàm của hướng điều khiển hay Góc đến (AoA) θ_0 , bước sóng của tần số sóng mang λ , và khoảng cách phần tử d . Tương tự như lọc theo thời gian, trọng số tạo búp sóng quiescent lý tưởng là bộ lọc phối hợp tương đương với vector đáp ứng không gian để triệt tiêu trực tiếp sự dịch pha biểu kiến trên mảng đối với mặt sóng tác động. Đáp ứng bộ lọc phối hợp tương đương với liên hợp phức của đáp ứng thu được:

$$w_n = s_n^* \quad (2.9)$$

Đáp ứng quiescent này cho $\theta = 0$ có thể thấy trong đồ thị bức xạ ở Hình 2.7; đồ thị bức xạ – đôi khi còn được gọi là mặt cắt phương vị cho ULA – được sử dụng để biểu diễn độ lợi mảng theo góc tới θ . Chúng ta thấy rằng thùy chính xuất hiện đúng tại góc 0, trong khi tất cả các góc khác đều bị suy hao.



[Hình 2.7: Đồ thị bức xạ chuẩn hóa của ULA, $\theta = 0$, $N = 16$]

Lưu ý rằng đôi khi hữu ích khi vẽ trục x của đồ thị bức xạ trong không gian sin, nơi góc được chuẩn hóa theo Phương trình 2.10 để biểu diễn góc tới dựa trên khoảng cách phần tử mảng và bước sóng tín hiệu đến:

$$\theta_{\text{Norm}} = (d/\lambda) \cdot \sin(\theta) \quad (2.10)$$

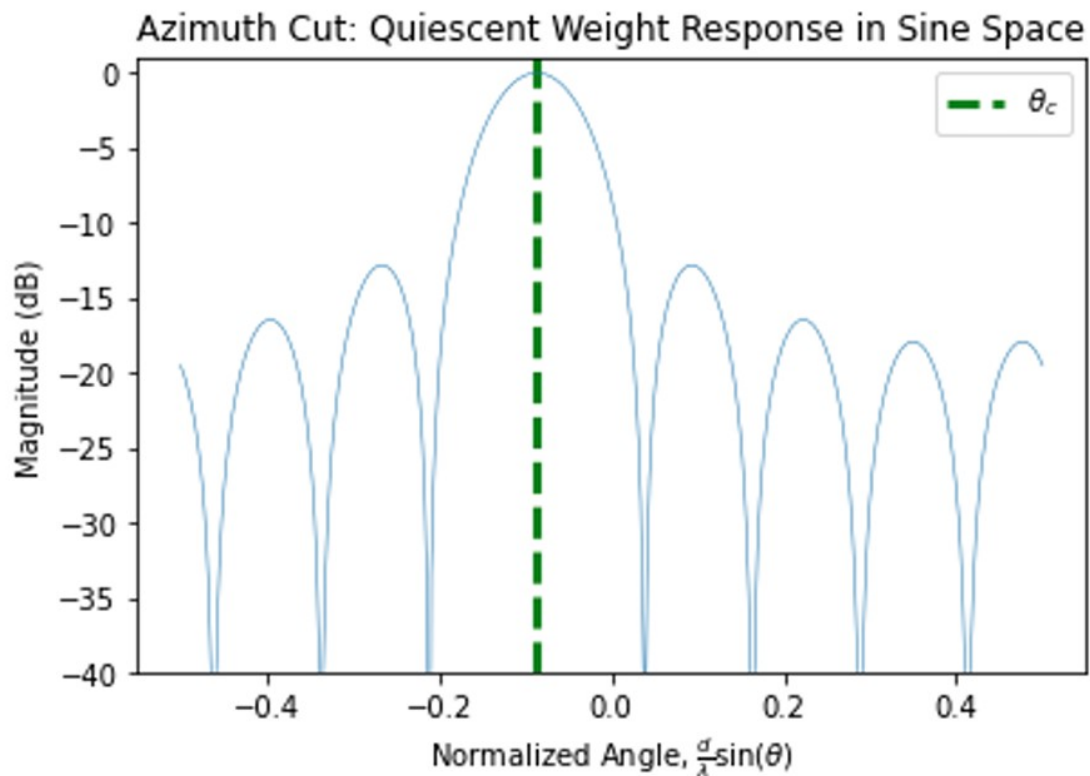
Đồ thị bức xạ này về cơ bản biểu diễn hệ số mảng (AF) của hệ thống, được định nghĩa trong Phương trình 2.11 là tổng đáp ứng điện áp của mảng dưới dạng tổng các đáp ứng điện áp riêng lẻ, dựa trên sự khác biệt giữa góc điều khiển ϕ và góc tới θ :

$$AF(\theta) = (1/N) \sum_{n=1}^N e^{-j2\pi(n-1)d/\lambda \cdot \sin(\theta_0 - \phi_n)} \quad (2.11)$$

Lưu ý sự tương đồng của số hạng phasor trong Phương trình 2.11 với vector đáp ứng không gian trong Phương trình 2.8 – đây là liên hợp phức của trọng số tạo búp sóng quiescent. Điều này có nghĩa là chúng ta có thể chèn trực tiếp các trọng số tạo búp sóng vào Phương trình 2.11 để tính toán hệ số mảng:

$$AF(\theta) = (1/N) \sum_{n=1}^N w_n \cdot e^{-j2\pi(n-1)d/\lambda \cdot \sin(\phi_n)} \quad (2.12)$$

Ví dụ, để tính toán các trọng số tạo búp sóng quiescent để điều khiển mảng đến -10° với khoảng cách nửa bước sóng, chúng ta đơn giản thay $\theta_0 = -10^\circ$ vào Phương trình 2.8 rồi tìm bộ lọc phối hợp tương đương trong Phương trình 2.9. Sau đó vẽ đáp ứng theo góc tới bằng Phương trình 2.12. Đồ thị không gian sin kết quả sẽ cho thấy đỉnh độ lợi tại $0.5 \cdot \sin(-10^\circ) \approx -0.09$.

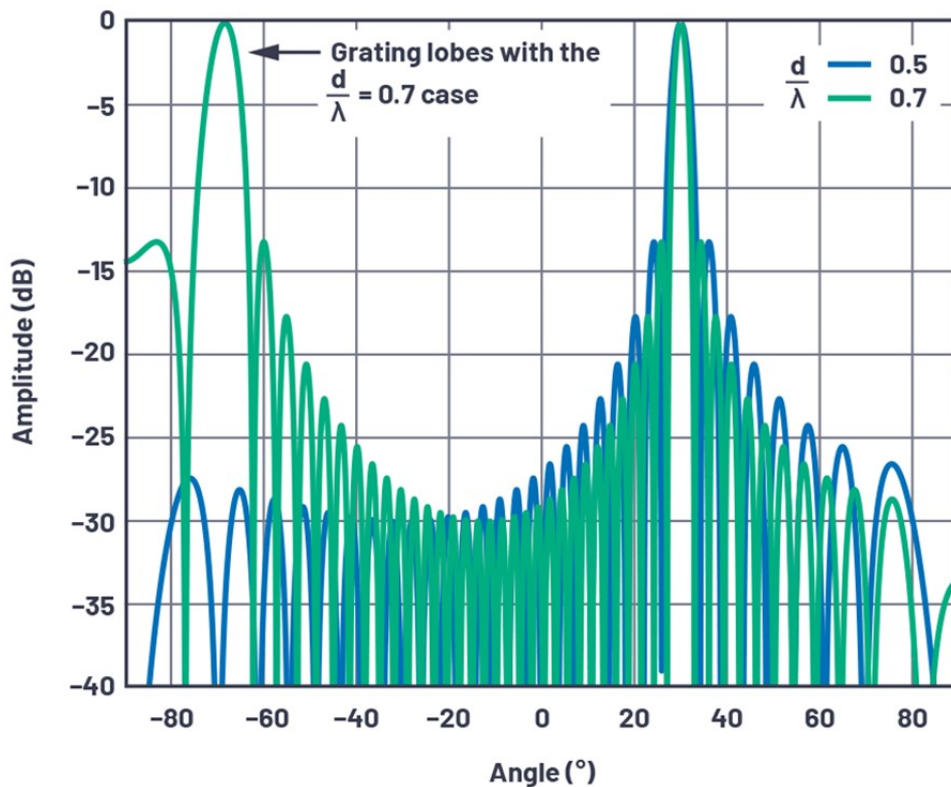


[Hình 2.8: Đồ thị không gian sin của trọng số quiescent]

2.2.3 Hiệu ứng mảng trong tạo búp sóng

Trong phần trước, chúng ta đã giới thiệu định lý lấy mẫu không gian cho các mảng pha trong đó yêu cầu khoảng cách phần tử $d < \lambda/2$. Nếu yêu cầu khoảng cách $d < \lambda/2$ bị vi phạm, đồ thị mảng của hệ thống tạo búp sóng sẽ xuất hiện các thùy nhám (grating lobes) – tương đương không gian với hiện tượng gập phổ (aliasing) trong miền thời gian bởi một hệ thống lấy mẫu thiếu, khi $B > f_s/2$. Các thùy nhám này xuất hiện như các vùng độ lợi trùng lặp trong đồ thị bức xạ của mảng.

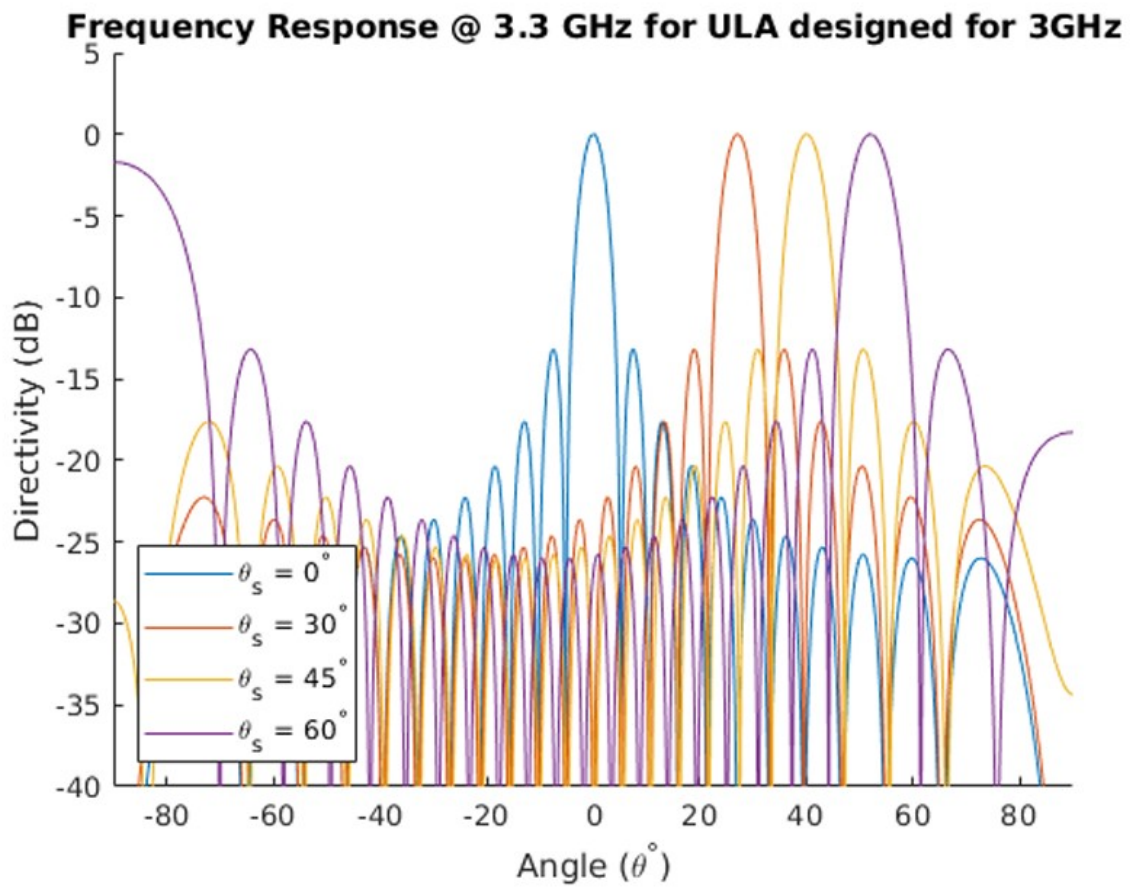
Khi quan sát Hình 2.9 cho ULA 32 kênh điều khiển tạo búp sóng ở góc tới 30° : khi $d/\lambda = 0.5$ (nửa bước sóng), chúng ta thấy đỉnh độ lợi mong đợi ở 30° và suy hao ở tất cả các hướng khác. Tuy nhiên, khi khoảng cách bước sóng lớn hơn nửa bước sóng ($d/\lambda = 0.7$), chúng ta bất ngờ thấy một đỉnh độ lợi khác ở khoảng -70° , điều này có thể gây vấn đề cho hệ thống tạo búp sóng khi nó chỉ mong đợi nhận tín hiệu từ một hướng cụ thể.



[Hình 2.9: Đồ thị bức xạ chuẩn hóa của ULA với khoảng cách phần tử khác nhau, $N = 32$]

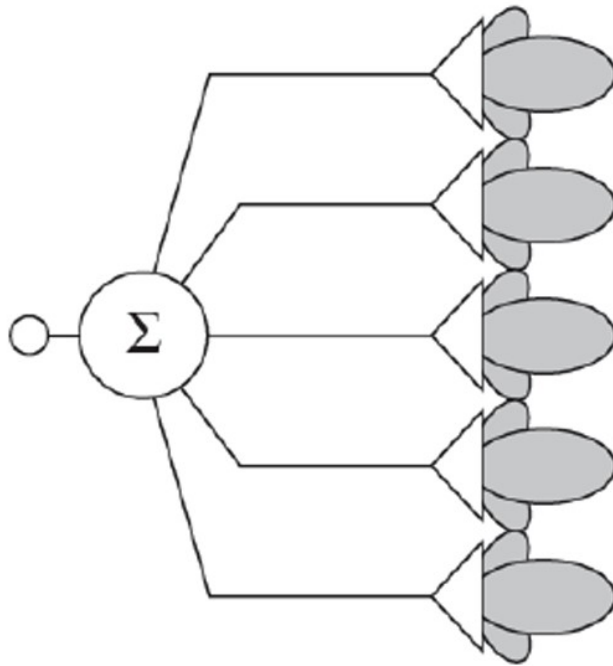
Đề cập trước đó, chúng ta giả định các hệ thống băng hẹp trong tài liệu này để đơn giản hóa toán học. Tuy nhiên, vấn đề có thể nảy sinh khi thực hiện tạo búp sóng băng hẹp – chẳng hạn như nhân với phasor phức để áp dụng dịch pha – khi các tín hiệu băng rộng hiện diện là hiện tượng "lệch búp sóng" (beam squint). Hình 2.10 cho thấy ULA được thiết kế cho tần số sóng mang 3 GHz nhận tín hiệu 3.3

GHz ở các góc tới khác nhau: khi góc tới di chuyển qua boresight ($\theta = 0$), thùy chính mở rộng và chúng ta thậm chí thấy một số gập phở vào các hướng tín hiệu khác ở các góc lớn hơn.



[Hình 2.10: Hiện tượng lệch búp sóng (Beam Squint)]

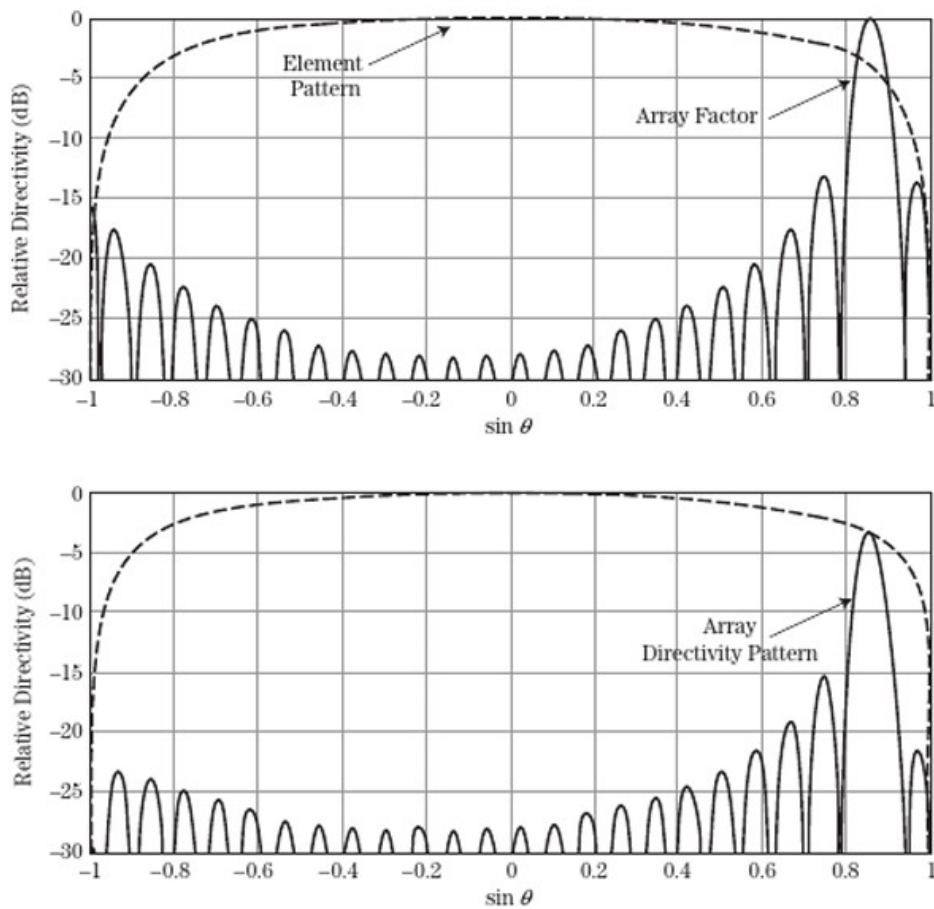
Một đơn giản hóa khác trong thảo luận là giả định rằng các phần tử mảng riêng lẻ là đẳng hướng. Tuy nhiên, các phần tử ăng-ten thực có đặc tính định hướng và đáp ứng góc riêng, như được biểu diễn trong Hình 2.11.



[Hình 2.11: Định hướng phần tử mảng riêng lẻ]

Do đó, một mô tả chính xác hơn về định hướng ăng-ten mảng thực tế $E(\theta)$ là kết hợp tuyến tính của hệ số mảng $AF(\theta)$ và định hướng phần tử riêng lẻ $E_e(\theta)$ như trong Phương trình 2.13:

$$E(\theta) = E_e(\theta) \cdot AF(\theta) \quad (2.13)$$

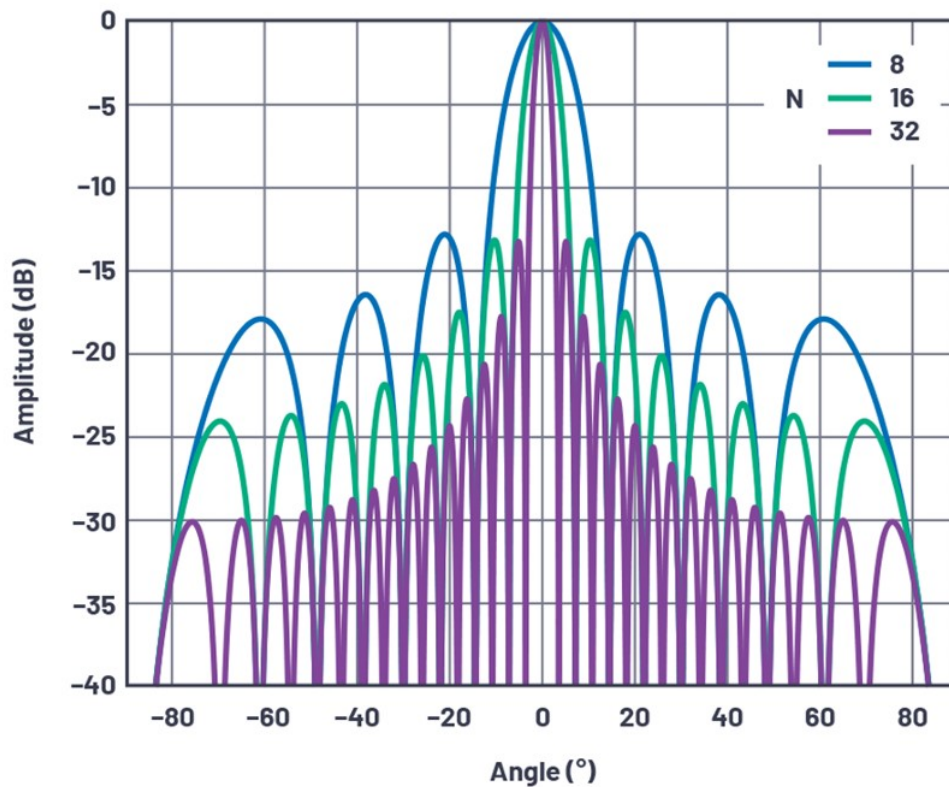


[Hình 2.12: Tổng định hướng ăng-ten mảng]

Do đó trong các hệ thống thực, chúng ta không thể luôn giả định rằng mỗi góc điều khiển đều cho thấy cùng một đáp ứng độ lợi. Cuối cùng, độ rộng thùy chính của hệ số mảng tỷ lệ nghịch với số lượng phần tử ăng-ten. Trong đáp ứng không gian hình chữ nhật chuẩn không có cửa sổ, độ rộng không đến không (null-to-null) của ULA có thể tìm theo Phương trình 2.14:

$$\theta_{_MB} = 2 \cdot \sin^{-1}[\lambda / (Nd) - \sin(\theta_0)] \quad (2.14)$$

Điều này có thể thấy trong Hình 2.13 nơi các hệ thống có số kênh cao hơn thấy độ rộng thùy chính hẹp hơn nhiều. Có vẻ hiển nhiên rằng thiết kế một hệ thống với số kênh cao hơn sẽ tạo ra một hệ thống tốt hơn, có tính định hướng cao hơn, tuy nhiên việc tăng số kênh RF làm tăng chi phí và độ phức tạp của hệ thống.

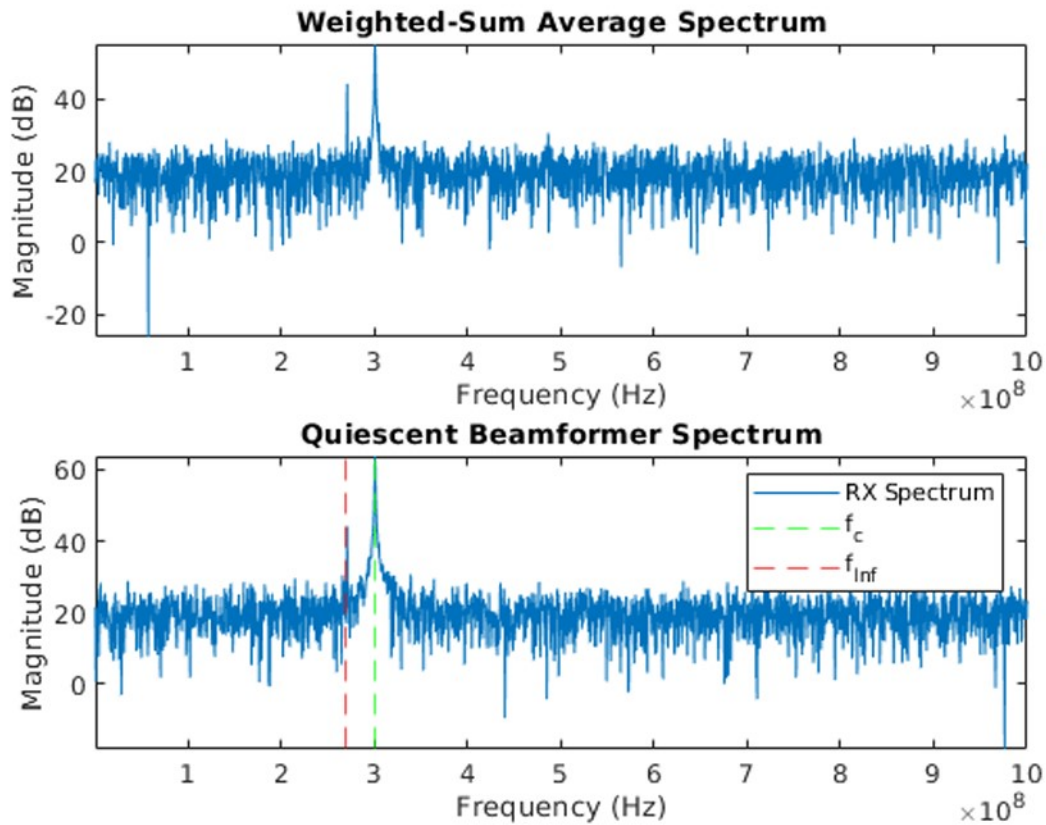


[Hình 2.13: Đồ thị bức xạ mảng với số lượng phần tử khác nhau, $d = \lambda/2$]

2.3 Tạo búp sóng thích nghi

Trong một số hệ thống, việc chỉ áp dụng trọng số quiescent để điều khiển mảng có thể không đủ, vì các tín hiệu từ các hướng không mong muốn vẫn có thể xâm nhập vào băng tần tín hiệu mong muốn, đặc biệt nếu có nhiều người dùng trong cùng băng này.

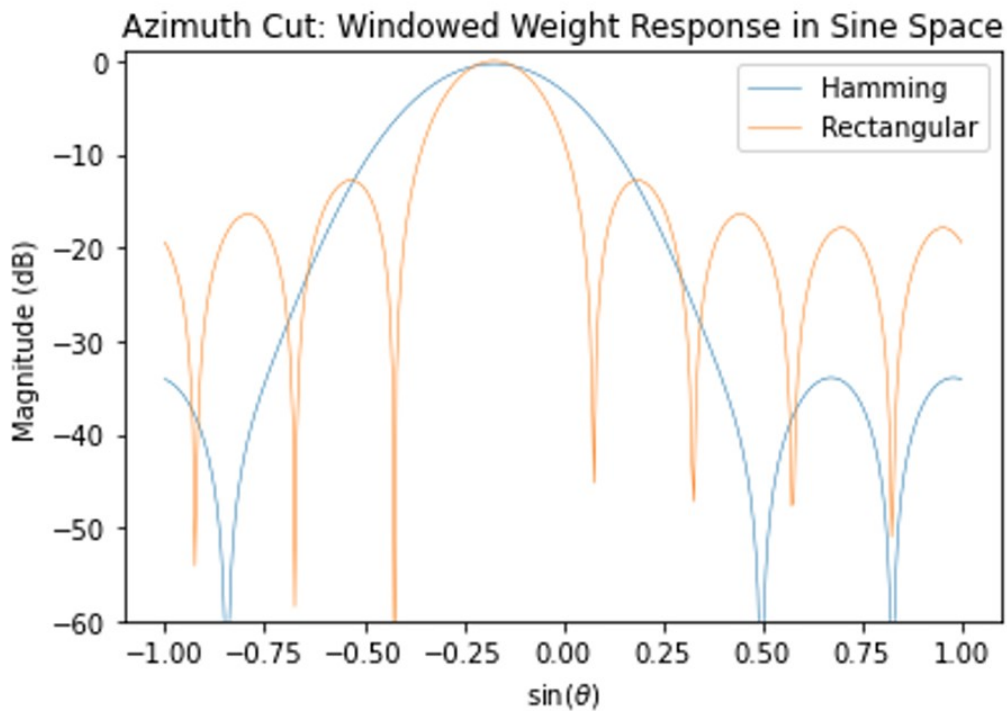
Ví dụ, trong trường hợp Hình 2.14, một tín hiệu mong muốn ở 300 MHz tác động lên mảng ở $\theta_d = 5^\circ$ và một nguồn nhiễu ở 270 MHz tác động ở $\theta_{Inf} = 30^\circ$. Cả trong trường hợp tổng có trọng số không tạo búp sóng lẫn trong trường hợp tạo búp sóng quiescent điều khiển đến $\theta_d = 5^\circ$, chúng ta vẫn thấy nguồn nhiễu trong phổ kết quả. Nguyên nhân là các thùy phụ của đáp ứng hệ số mảng không suy hao đủ các nguồn nhiễu; trong đáp ứng không cửa sổ hình chữ nhật, các thùy phụ gần nhất chỉ thấp hơn -13 dB so với thùy chính.



[Hình 2.14: Nhiễu quiescent, $d = \lambda/2$]

Một cách tiếp cận cơ bản để triệt tiêu các nguồn không mong muốn có thể là áp dụng cửa sổ (windowing) cho các phần tử ăng-ten riêng lẻ. Ví dụ, cửa sổ Hamming có thể được sử dụng để tạo các giá trị vô hướng để nhân với vector trọng số quiescent. Cửa sổ Hamming cung cấp các thùy phụ thấp hơn, đều hơn, nhưng với chi phí tăng độ rộng thùy chính, như thể hiện trong Hình 2.15:

$$w_{\text{hamming}}(n) = 0.54 - 0.46 \cdot \cos(2\pi n / (M-1)), \quad 0 \leq n \leq M-1 \quad (2.15)$$



[Hình 2.15: Đáp ứng không gian sin không của cửa sổ vs cửa sổ Hamming, $N = 8$]

Tuy nhiên, áp dụng cửa sổ đơn thuần vẫn có thể không lý tưởng vì độ rộng thùy chính tăng có nghĩa là các nguồn nhiễu gần hướng không gian của nguồn mong muốn có độ lợi gần như bằng nhau. Thêm nữa, dù các thùy phụ bị suy hao nhiều hơn trong các đáp ứng có cửa sổ, một nguồn nhiễu rất mạnh vẫn có thể xuất hiện trong phổ đầu ra nếu nguồn phát này không rơi vào một null tự nhiên.

Cũng cần lưu ý rằng sự khác biệt về tần số giữa tín hiệu mong muốn và tín hiệu nhiễu trong các ví dụ trên chủ yếu là để dễ dàng minh họa. Hai tín hiệu thực ra có thể ở cùng tần số, ví dụ hai người dùng truyền thông chiếm cùng một kênh, trong đó cả hai tín hiệu có thể trực tiếp gây nhiễu lẫn nhau. Giả định duy nhất là cả tín hiệu nhiễu và tín hiệu mong muốn đều phát ra từ các vị trí không gian khác nhau, điều này tạo nền tảng cho xử lý như Tạo búp sóng thích nghi để tách tín hiệu mong muốn khỏi nhiễu và tạp âm.

2.3.1 Tạo búp sóng thích nghi MVDR

Chức năng cơ bản của Tạo búp sóng thích nghi là tính toán các trọng số tạo búp sóng để cung cấp độ lợi cho các tín hiệu tới từ hướng mong muốn, trong khi điều chỉnh linh hoạt để tạo các null không gian triệt tiêu tín hiệu từ các vị trí không gian khác. Hiệu ứng triệt tiêu này được thực hiện bằng cách đặt một null không gian – vùng có suy hao lớn – theo các hướng có nguồn nhiễu, đây cũng là lý do Tạo búp sóng thích nghi còn được gọi là điều khiển null.

Mặc dù có nhiều thuật toán và triển khai khác nhau, thuật toán Đáp ứng không méo phương sai cực tiểu (MVDR) là một phương pháp cổ điển, phụ thuộc dữ liệu cho tạo búp sóng thích nghi. MVDR có ưu điểm là tốc độ hội tụ nhanh và khả năng xử lý nhiều nguồn nhiễu phức tạp. Phương pháp MVDR còn được gọi trong tài liệu là phương pháp Capon (Capon, 1969).

Phương pháp MVDR hoạt động trên một batch dữ liệu mẫu trên mỗi kênh không gian tại một thời điểm nhất định, ở đây gọi là snapshot gồm K mẫu. Ma trận dữ liệu mẫu $N \times K$ là X , với N hàng (mỗi hàng tương ứng với một phần tử mảng riêng biệt) gồm K mẫu dữ liệu:

$$X_{N,K} = [[x_{1,1} \ x_{1,2} \dots x_{1,K}], \dots, [x_{N,1} \ x_{N,2} \dots x_{N,K}]] \quad (2.16)$$

Từ ma trận dữ liệu đầu vào X , ma trận hiệp phương sai mẫu ước lượng M có thể được hình thành. Giả định cho các mục đích thực tế là chúng ta đang xử lý các hệ thống quá định với số mẫu mỗi kênh K lớn hơn số kênh N . Do đó, M là ma trận $N \times N$ được hình thành bởi kỳ vọng $E(\cdot)$ của tích ma trận:

$$M = E[X \cdot X^H] \quad (2.17)$$

Cho các mẫu đầu vào trung bình không – như giả định cho hầu hết các hệ thống RF – ma trận hiệp phương sai M tương đương với ma trận tự tương quan. Hơn nữa, các nguồn không tương quan lẫn nhau trong X có nghĩa là mỗi mẫu có xác suất bằng nhau, do đó kỳ vọng $E(\cdot)$ của tích ma trận về cơ bản là ma trận tương quan trung bình thời gian:

$$M = (1/K) \sum_{n=1}^K x^{(n)} x^{(n)H} \quad (2.18)$$

Cũng cần lưu ý rằng ma trận hiệp phương sai về cơ bản bao gồm ma trận hiệp phương sai tín hiệu quan tâm (SOI) M_d và ma trận hiệp phương sai nhiễu cộng tạp âm M_{i+n} :

$$M = M_d + M_{i+n} \quad (2.19)$$

Thước đo hiệu suất cho thuật toán tạo búp sóng thích nghi là mức độ tăng cường công suất tín hiệu SOI từ hướng điều khiển mong muốn trong khi suy hao các nguồn tạp âm và nhiễu từ các hướng khác; về mặt toán học điều này được biểu diễn bằng tỷ lệ SINR:

$$\text{SINR} = P / (I + N) \quad (2.20)$$

SINR cũng là thước đo quý giá cho cả hệ thống MIMO và không MIMO. Giá trị SINR tuyến tính (không dB) có thể được sử dụng để ước lượng giới hạn lý thuyết trên của dung lượng kênh truyền thông (C bit/giây), cho băng thông kênh B Hz, theo định lý Shannon-Hartley:

$$C = B \cdot \log_2(1 + \text{SINR}) \quad (2.21)$$

Cho các ma trận hiệp phương sai $N \times N$ cho SOI và nhiễu cộng tạp âm được định nghĩa ở trên, và một vector trọng số tạo búp sóng $N \times 1$ w nào đó, chúng ta thực sự có thể trực tiếp tính toán SINR dự kiến theo Phương trình 2.22:

$$\text{SINR} = (\mathbf{w}^H \cdot \mathbf{M}_d \cdot \mathbf{w}) / (\mathbf{w}^H \cdot \mathbf{M}_{\{i+n\}} \cdot \mathbf{w}) \quad (2.22)$$

Cũng cần thiết cho phương pháp MVDR phụ thuộc dữ liệu, $\mathbf{s}(\theta)$ là vector điều khiển $N \times 1$, tương đương với Phương trình 2.23 cho góc điều khiển mảng mong muốn θ :

$$\mathbf{s}_n(\theta) = [1, e^{-j\theta}, \dots, e^{-j(N-1)\theta}]^T \quad (2.23)$$

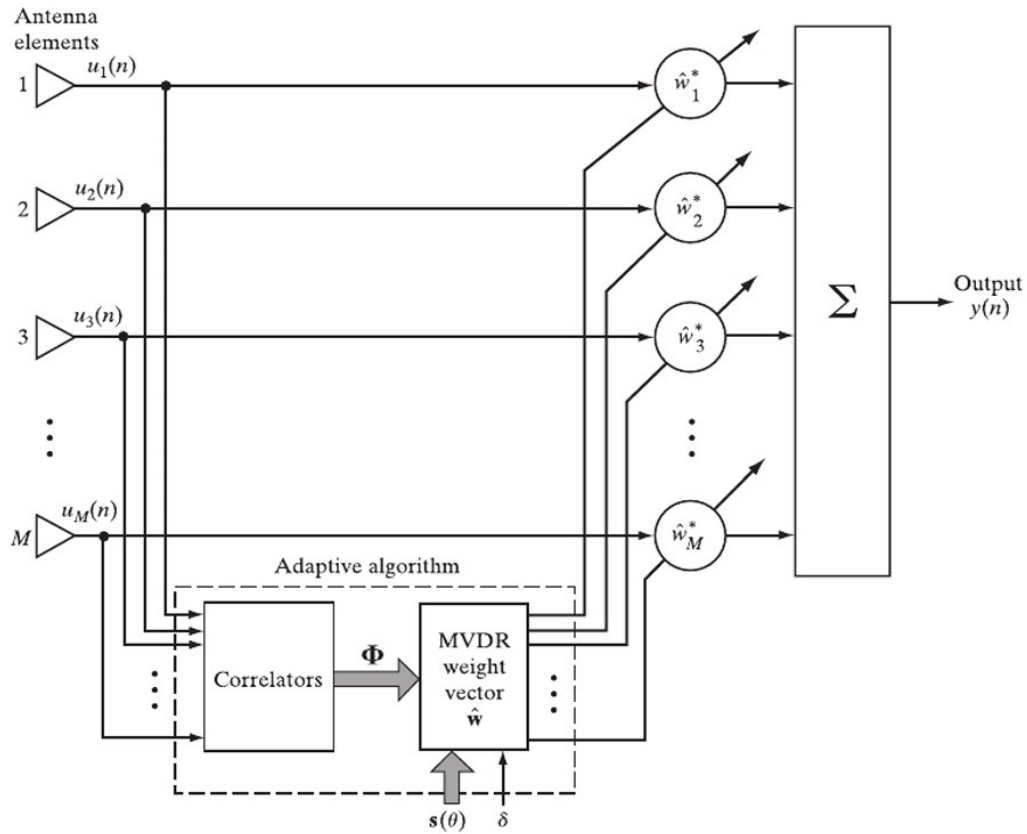
Lưu ý rằng vector điều khiển tương đương với liên hợp phức của vector đáp ứng không gian được định nghĩa trong Phương trình 2.8 trong phần trước. Do đó, bộ tạo búp sóng tối ưu tối thiểu hóa công suất đầu ra dưới ràng buộc $\mathbf{w}^H \cdot \mathbf{s}(\theta) = 1$ thông qua phương trình tối thiểu hóa trong Phương trình 2.24:

$$\hat{\mathbf{w}} = \min_{\mathbf{w}} \mathbf{w}^H \cdot \mathbf{M} \cdot \mathbf{w} \quad (2.24)$$

Từ đây chúng ta có thể đưa ra định nghĩa toán học của giải pháp MVDR cho vector trọng số tạo búp sóng thích nghi $N \times 1$ $\hat{\mathbf{w}}$ trong Phương trình 2.25:

$$\hat{\mathbf{w}} = \mathbf{M}^{-1} \cdot \mathbf{s}(\theta) / (\mathbf{s}^H(\theta) \cdot \mathbf{M}^{-1} \cdot \mathbf{s}(\theta)) \quad (2.25)$$

Một ví dụ về bộ tạo búp sóng MVDR có thể thấy trong Hình 2.16, trong đó ma trận tương quan/hiệp phương sai được xây dựng từ snapshot K mẫu dữ liệu từ mỗi kênh, và được đưa vào bộ xử lý MVDR để tính toán các trọng số tạo búp sóng thích nghi được áp dụng qua từng kênh, sau đó cộng kết hợp để tạo ra búp sóng đầu ra $y(n)$.

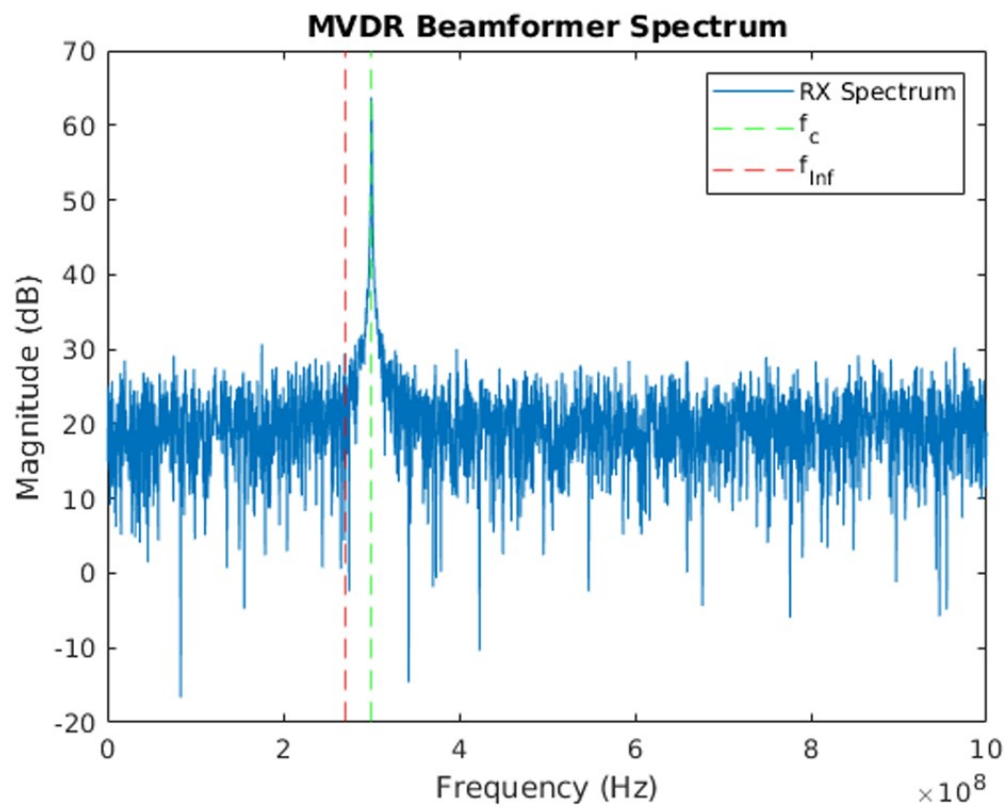


[Hình 2.16: Sơ đồ khối bộ tạo búp sóng MVDR cho ULA]

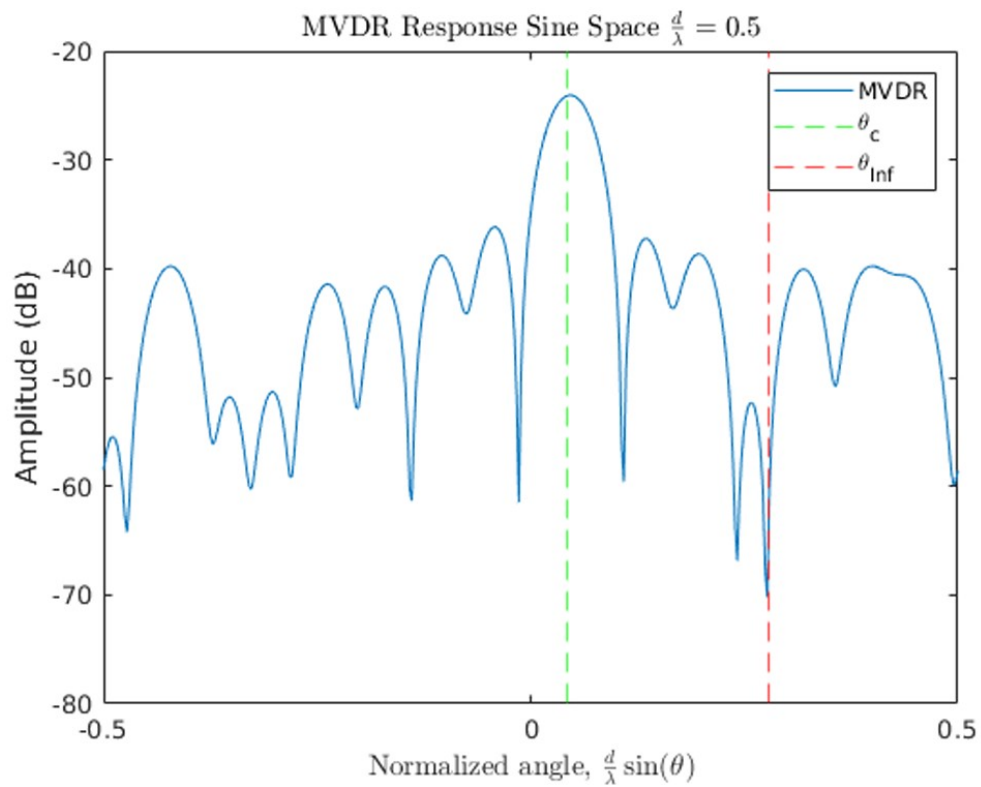
Sử dụng cùng ví dụ từ phần tạo búp sóng tắt định, chúng ta có thể áp dụng các trọng số tạo búp sóng được tính bởi MVDR. Tín hiệu mong muốn ở 300 MHz tác động mảng tại $\theta_d = 5^\circ$ và nguồn nhiễu ở 270 MHz tác động tại $\theta_{Inf} = 30^\circ$. Đầu ra tạo búp sóng bằng hợp MVDR được cho bởi:

$$y(k) = \hat{w}^H \cdot x(k) \quad (2.26)$$

Như thấy trong Hình 2.17, tín hiệu SOI mong muốn có SINR rất cao do độ lợi tạo búp sóng được áp dụng, và tín hiệu nhiễu không còn xuất hiện trong phổ đầu ra. Việc nguồn nhiễu đã thực sự bị triệt tiêu cũng có thể thấy qua Hình 2.18, biểu diễn đồ thị bức xạ của trọng số MVDR trong không gian sin, hướng nhiễu thấy một null sâu.



[Hình 2.17: Phổ đầu ra từ bộ tạo búp sóng MVDR cho ULA]

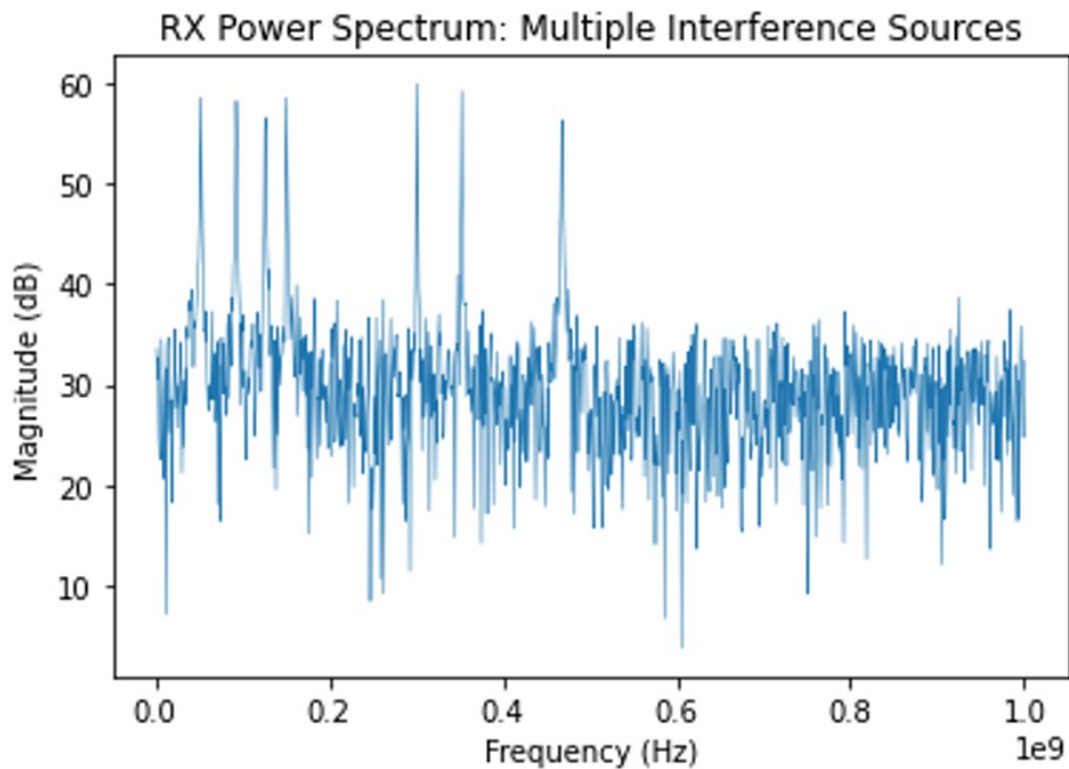


[Hình 2.18: Đồ thị bức xạ của trọng số tạo búp sóng MVDR]

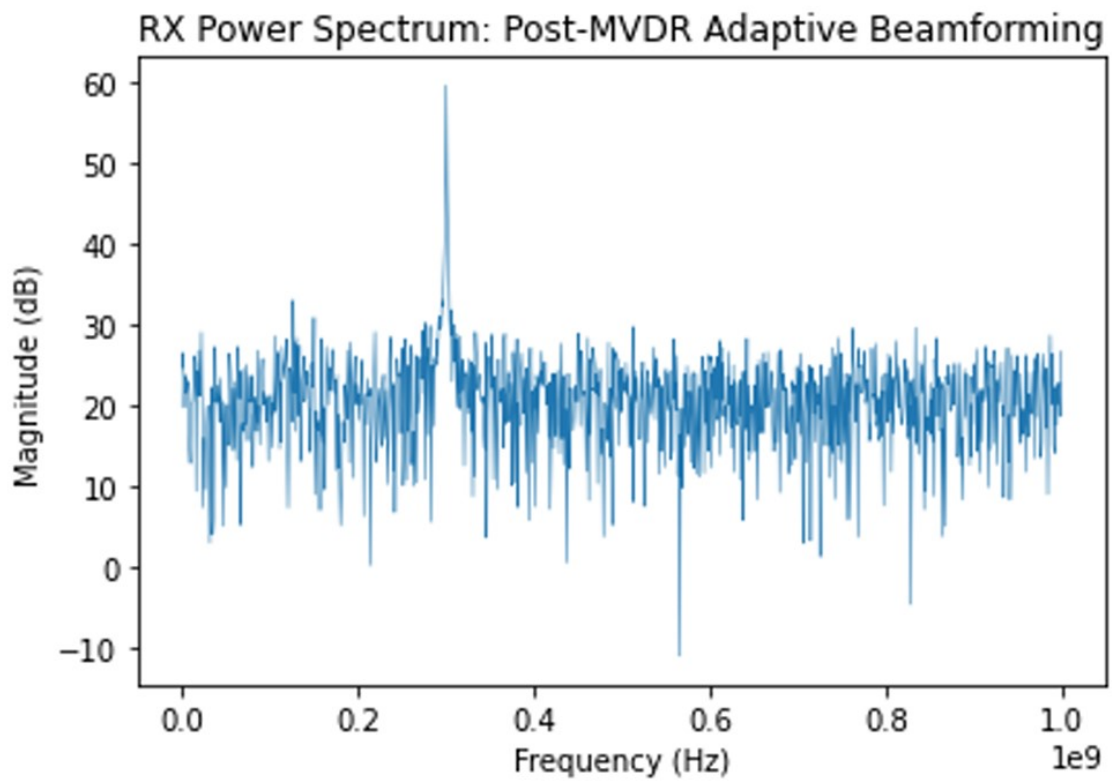
Dựa trên đáp ứng không gian của một số lượng phần tử ăng-ten nhất định, bậc tự do (DOF) của mảng N phần tử về cơ bản được thúc đẩy bởi số lượng null độc lập có thể được tạo ra bởi thuật toán MVDR, như định nghĩa trong Phương trình 2.27:

$$\text{DOF} = N - 2 \quad (2.27)$$

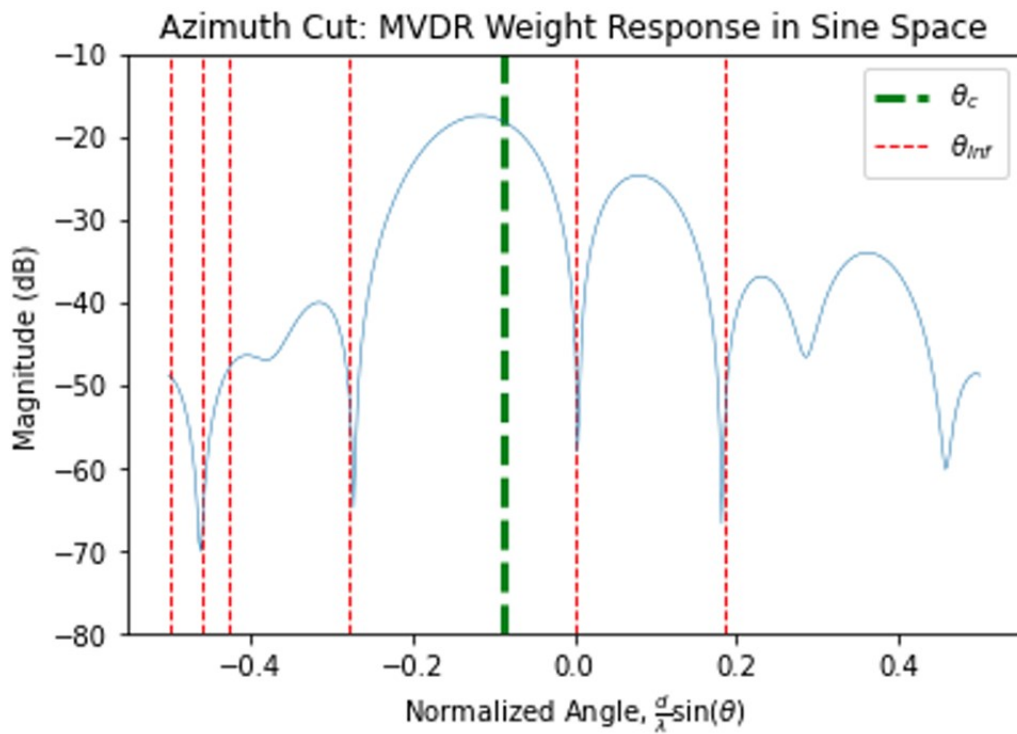
Điều này có nghĩa là có thể triệt tiêu tới $N-2$ nguồn nhiễu bằng MVDR. Để minh họa điều này trong thực tế, một ULA 8 phần tử có thể được chứng minh có 6 nguồn nhiễu, ở các tần số băng hẹp thay đổi và các góc tới thay đổi. Sau tạo búp sóng thích nghi MVDR, phổ đầu ra trong Hình 2.20 được làm sạch chỉ còn tín hiệu mong muốn từ hướng mong muốn. Một lần nữa trong không gian sin (Hình 2.21), chúng ta có thể thấy các hướng nhiễu không gian khác nhau được triệt tiêu thích nghi.



[Hình 2.19: Phổ đầu ra với nhiễu nhiều nguồn, $N = 8$]

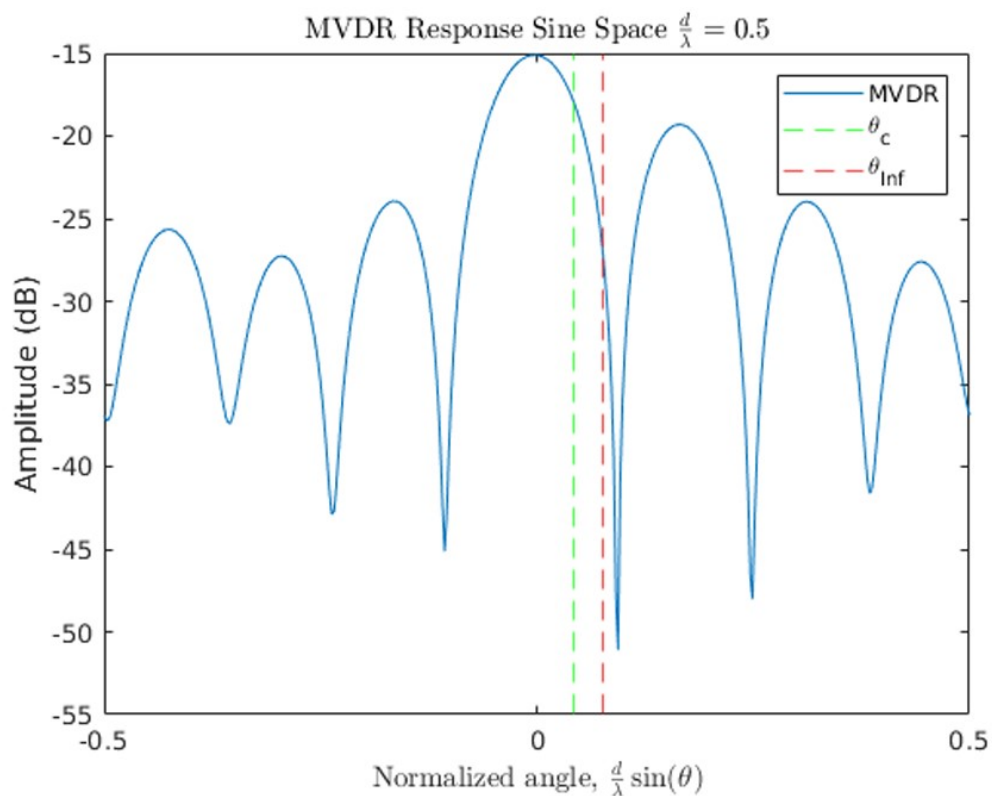


[Hình 2.20: Phổ đầu ra MVDR với nhiều nguồn nhiễu, $N = 8$]



[Hình 2.21: MVDR với nhiều nguồn nhiễu trong không gian sin, $N = 8$]

Lưu ý rằng trong một số trường hợp, MVDR không thể triệt tiêu hoàn hảo tất cả các nguồn nhiễu. Thông thường điều này là do vị trí không gian tương đối của các nguồn nhiễu so với hướng mong muốn. Hình 2.22 cho thấy ULA $N = 8$ kênh với tín hiệu mong muốn tác động tại $\theta_d = 5^\circ$ nhưng nguồn nhiễu ở rất gần về không gian, tác động tại $\theta_{\text{Inf}} = 8^\circ$. Thuật toán đang cố gắng đẩy hướng nhiễu vào một null trong khi vẫn duy trì độ lợi theo hướng điều khiển mong muốn, tuy nhiên độ rộng thùy chính của ULA 8 phần tử quá rộng để thực hiện cả hai. Điều này cho thấy rằng nhiều phần tử ăng-ten không chỉ cung cấp nhiều null hơn mà còn tạo ra các thùy hẹp hơn, giúp triệt tiêu dễ dàng hơn các hướng nhiễu gần nhau.



[Hình 2.22: MVDR với nguồn nhiễu quá gần trong không gian sin, $N = 8$]

Chương 3: Học máy ứng dụng vào tạo búp sóng thích nghi

Thay vì đi theo hướng tiếp cận truyền thống là triển khai giải pháp dạng đóng từ các thuật toán tối ưu hóa thích nghi kinh điển (như IQRD-RLS) vốn đòi hỏi cấu trúc phần cứng phức tạp và khối lượng tính toán lớn, chúng ta có thể khám phá tiềm năng của việc ứng dụng những tiến bộ trong lĩnh vực học máy để tạo ra các trọng số thích nghi này theo một cách tiếp cận hiệu quả hơn.

Cách tiếp cận được lựa chọn là lấy ma trận hiệp phương sai và vector điều khiển làm lớp đầu vào hai chiều và tạo ra trực tiếp các trọng số thích nghi ở đầu ra của Mạng nơ-ron tích chập (CNN). Phần lớn tài liệu tham khảo về CNN trong RF tập trung vào phân loại tín hiệu. Trường hợp tạo búp sóng thích nghi đòi hỏi CNN thực hiện hồi quy đa đầu ra, không phải phân loại. Theo tham khảo công trình gần nhất với cách tiếp cận của chúng ta là nơi các tác giả sử dụng CNN cho tạo búp sóng thích nghi trong ứng dụng siêu âm, cho thấy hiệu quả xử lý 96.4%.

4.1 Nền tảng về Học sâu

Học sâu, và rộng hơn là lĩnh vực Học máy, cực kỳ đa dạng và kéo từ nhiều ngành toán học và khoa học. Hơn nữa, lĩnh vực này đang phát triển từng ngày. Trong những năm gần đây, nhờ sự gia tăng mạnh mẽ về dữ liệu, năng lực tính toán và các thuật toán tối ưu hóa hiệu quả hơn, các mô hình học sâu đã đạt được nhiều kết quả nổi bật trong các bài toán như phân loại, dự đoán, nhận dạng mẫu và xử lý tín hiệu. Đặc biệt, với khả năng tự động học đặc trưng từ dữ liệu thô, học sâu đã dần thay thế nhiều phương pháp truyền thống vốn phụ thuộc nhiều vào việc thiết kế đặc trưng thủ công.

4.1.1 Kiến trúc CNN

Một kiến trúc phổ biến cho các mạng nơ-ron sâu là Mạng nơ-ron tích chập (CNN). Cơ sở của CNN là nó mô hình hóa sự tương tác của các nơ-ron trong não người, trong đó một tập hợp dữ liệu đầu vào x được tích chập với một tập hợp trọng số – còn gọi là kernel – để tạo ra bản đồ đặc trưng đầu ra. Phép tích chập rời rạc tạo nên cơ sở của CNN:

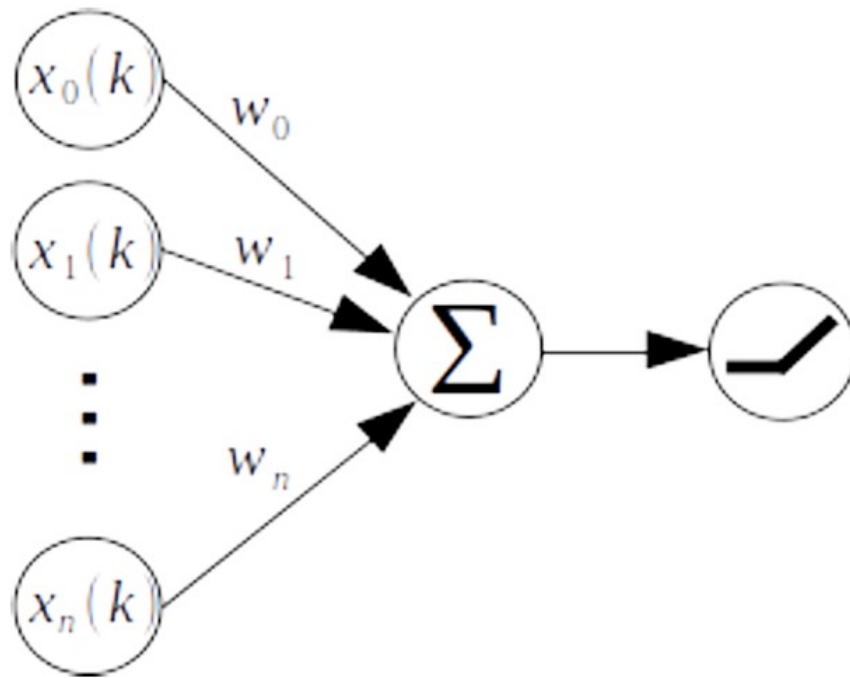
$$s(t) = (x * w)(t) = \sum(a) x(a) \cdot w(t-a) \quad (4.1)$$

Đối với đầu vào $m \times n$ hai chiều, phép tích chập 2D cũng có thể được sử dụng:

$$S(i,j) = (X * K)(i,j) = \sum \sum X(i-m, j-n) \cdot K(m,n) \quad (4.2)$$

Xét ở cấp độ vi mô, bản chất của phép toán tích chập này được xây dựng dựa trên nguyên lý hoạt động của một nơ-ron độc lập (perceptron). Tại mỗi cửa sổ trượt trên không gian hai chiều, một vùng dữ liệu đầu vào sẽ được nhân chập với ma trận trọng số (kernel) tương ứng, sau đó toàn bộ các tích này

được cộng tổng lại với nhau – bản chất chính là thực hiện phép nhân vô hướng (dot product) giữa hai tập hợp số. Kết quả sau phép cộng tổng dịch vector này sẽ được đưa qua một hàm kích hoạt (activation function). Hàm kích hoạt này không chỉ đóng vai trò mô phỏng lại cơ chế kích phát sinh học của các nơ-ron thực tế, mà còn là thành phần cốt lõi tạo ra tính phi tuyến cho mô hình, giúp mạng CNN có thể tối ưu hóa và cập nhật trọng số một cách hiệu quả trong quá trình huấn luyện.

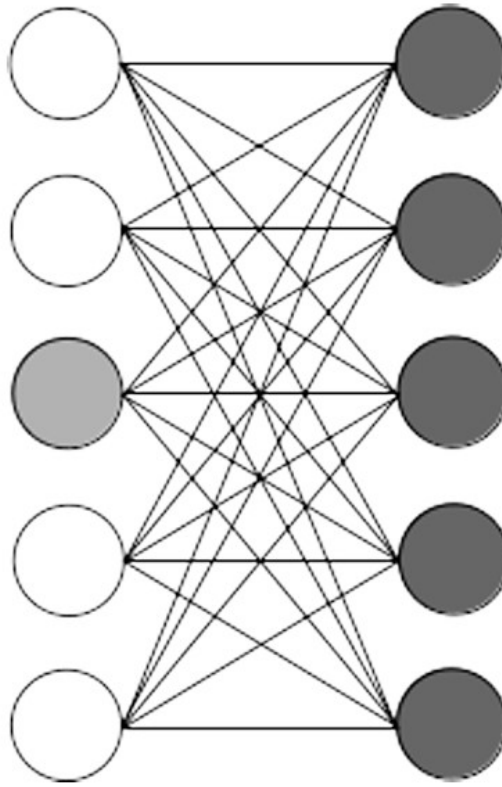


[Hình 4.1: Khởi xây dựng Perceptron trong Mạng nơ-ron]

Hàm kích hoạt Đơn vị tuyến tính chỉnh lưu (ReLU) phổ biến đã được sử dụng cho nghiên cứu này; ReLU không chỉ mô phỏng nơ-ron sinh học mà còn được chứng minh cho phép huấn luyện tốt hơn các mạng nơ-ron sâu so với các hàm kích hoạt sigmoid logistic hoặc tiếp tuyến hyperbolic trước đây:

$$\text{ReLU}(x) = x^+ = \max(0, x) \quad (4.3)$$

Kết nối đầy đủ của một số lượng đầu vào với một số lượng nơ-ron, nơi mỗi nơ-ron được cung cấp tất cả các đầu vào, được gọi là lớp kết nối đầy đủ (FC) trong CNN. Lớp FC có tính chất là sự thay đổi trong một giá trị đầu vào có tác động tương ứng đến tất cả các giá trị đầu ra của lớp tiếp theo, như thể hiện trong Hình 4.2. Sự kết hợp tuần tự của nhiều lớp tạo thành mạng và là cơ sở của CNN tổng thể.



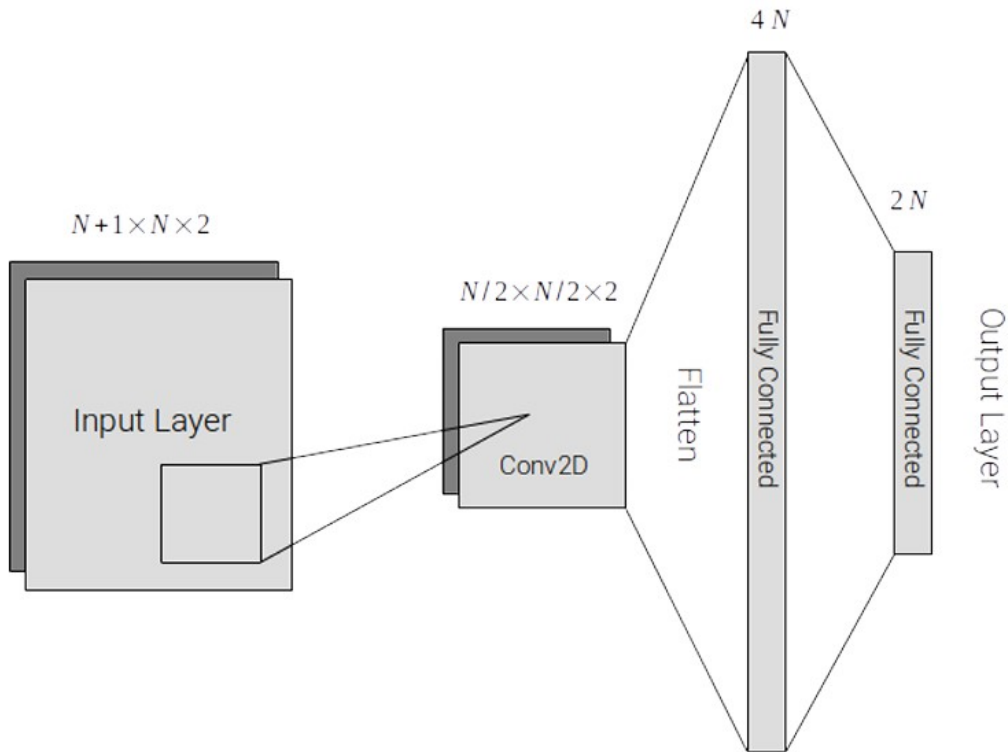
[Hình 4.2: Lớp kết nối đầy đủ]

4.1.2 Phát triển mô hình CNN trong TensorFlow

Bắt đầu với lớp đầu vào của mô hình CNN, quyết định được đưa ra là sử dụng ma trận hiệp phương sai đã tạo như một bước tiền xử lý trước CNN. Điều này vừa tương đối dễ triển khai trong logic FPGA vừa nhẹ về tài nguyên. Nếu cách tiếp cận chuỗi thời gian được chọn, một cấu trúc NN khác sẽ được sử dụng, chẳng hạn như mạng Long Short-Term Memory (LSTM).

Vector điều khiển được bao gồm như một phần của tập dữ liệu đầu vào của CNN, tương tự như các thuật toán tạo búp sóng thích nghi trước đó, bằng cách nối thêm vào ma trận hiệp phương sai đầu vào như một hàng bổ sung. Vì ma trận hiệp phương sai có giá trị phức và hầu hết các công cụ CNN giả định dữ liệu thực, chúng ta định nghĩa lớp đầu vào là 2D với kích thước $(N+1) \times N \times 2$ – một kênh cho phần thực, một cho phần ảo.

Lớp đầu ra được thiết kế có kích thước 2N: phần thực của trọng số tạo búp sóng thích nghi trong nửa dưới của vector đầu ra, và phần ảo trong nửa trên. Qua thực nghiệm, cấu trúc CNN được chọn với một lớp tích chập 2D, một lớp ẩn phẳng và một lớp kết nối đầy đủ cuối cùng. Cấu trúc này được hiển thị trong Hình 4.3.



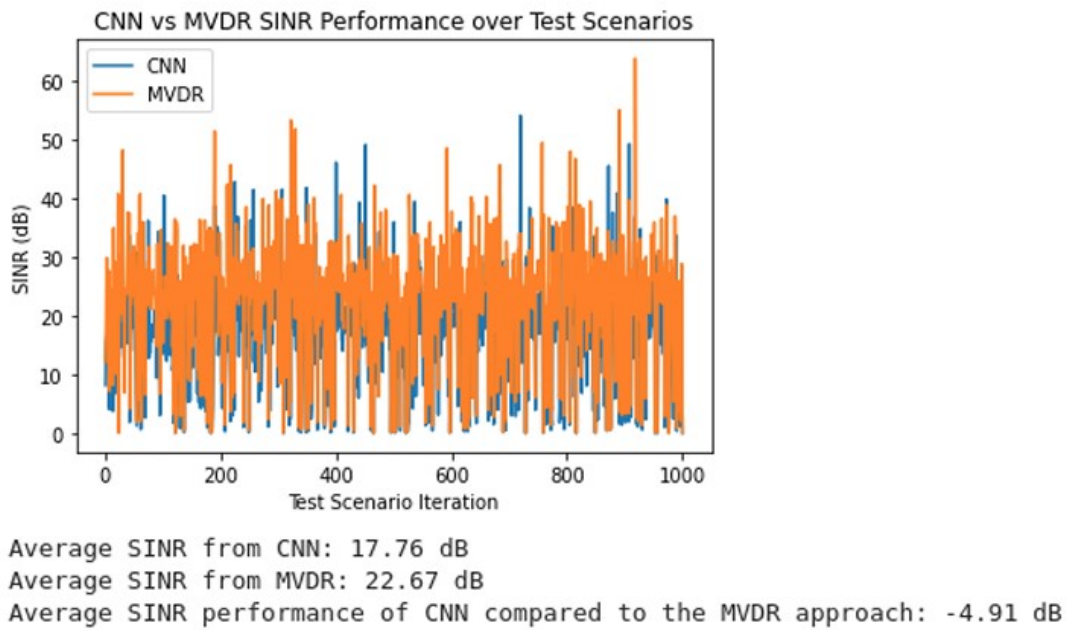
[Hình 4.3: Cấu trúc CNN Tạo búp sóng thích nghi]

Sau khi định nghĩa mô hình CNN, chúng ta tiến hành huấn luyện mô hình. Để thực hiện điều này, một tập dữ liệu kiểm tra tổng hợp được tạo trong Python để tạo ra số lượng lớn các kịch bản kiểm tra. Mỗi kịch bản được tạo ra để thay đổi: số lượng nguồn nhiễu (lên đến giới hạn lý thuyết $N-1$ cho triệt tiêu), góc tới của nhiễu và SOI, tần số trung tâm, và tỷ lệ tín hiệu trên tạp âm (SNR). Số lượng lớn kịch bản kiểm tra và số lượng lớn tham số thay đổi nhằm tránh vấn đề overfitting trong quá trình huấn luyện mô hình.

Trong học có giám sát, chúng ta cung cấp dữ liệu đầu vào kiểm tra cho mạng, trong đó mỗi kịch bản có một nhãn liên quan, và kiểm tra đầu ra của mạng xem nó suy diễn gần đến nhãn mong đợi đến mức nào. Sai số thống kê giữa tập giá trị suy diễn (Y) và mong đợi ($\hat{Y}$) có thể được tính bằng Sai số bình phương trung bình (MSE):

$$MSE = (1/n) \sum_{i=1 \text{ đến } n} (Y_i - \hat{Y}_i)^2 \quad (4.4)$$

Trong công trình này, chúng ta sử dụng MSE để tính hàm mất mát trong quá trình huấn luyện giữa các trọng số suy diễn trong một batch huấn luyện và tập trọng số lý tưởng từ quá trình tính toán MVDR chuẩn. Tốt hơn sẽ là tạo hàm mất mát tùy chỉnh dựa trên tính toán $1/\text{SINR}$, tuy nhiên TensorFlow hiện không hỗ trợ các hàm mất mát có giá trị phức. Do đó, sự khác biệt SINR trung bình là gần 5 dB kém hơn từ triển khai CNN so với MVDR, như thể hiện trong Hình 4.4.



[Hình 4.4: CNN so với MVDR SINR trên tập dữ liệu xác thực]

Nguồn ảnh: [33] J. Gentile, “FPGA-Based Adaptive Digital Beamforming Using Machine Learning for MIMO Systems,” , May 2021.

Sự phát triển TensorFlow trong tương lai, hoặc có thể sử dụng bộ công cụ khác hỗ trợ các phép tính có giá trị phức, nên được theo đuổi cho công việc tương lai để phát triển tốt hơn mạng nơ-ron cho tạo búp sóng thích nghi. Tuy nhiên, hiệu suất thấp hơn nhẹ này vẫn đủ để chứng minh khái niệm.

4.2 Triển khai CNN trên FPGA

Bây giờ khi đã có CNN hoạt động được huấn luyện để thực hiện tạo búp sóng thích nghi, chúng ta có thể chuyển các lớp và cấu trúc sang logic FPGA. Tuy nhiên, trước khi chuyển CNN, chúng ta nên thực hiện một số bước để tối ưu hóa mô hình cho các mục tiêu nhưng như FPGA sử dụng tiện ích TensorFlow Lite.

Đầu tiên, có thể thực hiện thao tác được gọi là cắt tỉa (pruning) để loại bỏ (đặt về không) các nút trong mô hình có ít ảnh hưởng đến độ chính xác tổng thể của mô hình. Quan trọng hơn là quá trình lượng tử hóa (quantization). Lượng tử hóa là quá trình chuyển đổi trọng số mô hình từ biểu diễn số chính xác đầy đủ – thường là dấu phẩy động 32-bit – sang biểu diễn số nhỏ hơn như số nguyên, điều này thích hợp hơn cho logic FPGA. Chế độ "lượng tử hóa 16x8" được sử dụng trong đó trọng số được lượng tử hóa về giá trị số nguyên 8-bit trong khi các kích hoạt được chuyển đổi sang giá trị 16-bit, cung cấp phạm vi động tốt hơn trong khi vẫn giảm đáng kể việc sử dụng tài nguyên. Lượng tử hóa một tập hợp

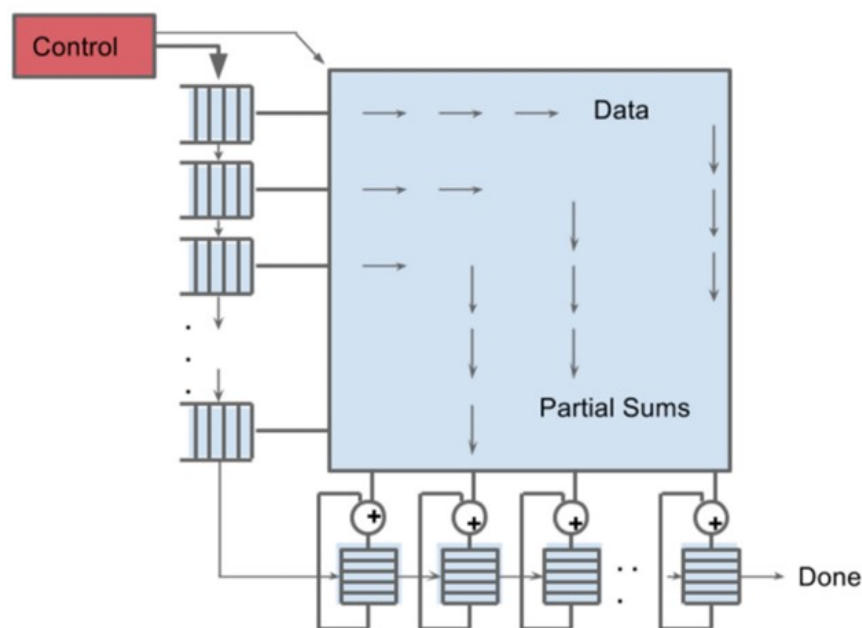
các giá trị dấu phẩy động b sang kiểu dữ liệu số nguyên n-bit có dấu được thực hiện bằng cách nhân mỗi giá trị với hệ số tỷ lệ k:

$$k = (2^{(n-1)} - 1) / \max|x|, \quad x \in b \quad (4.5)$$

Bắt đầu với khối xây dựng perceptron cơ bản, một thành phần SystemVerilog được tạo ra thực hiện phép tính tích chấm. Thay vì tính toán tất cả các tích trọng số kết nối song song, chúng ta sử dụng một bộ nhân duy nhất trong perceptron và lặp qua từng kết nối, nhân với trọng số liên quan được lưu trong ROM cục bộ. Với CNN có hơn 2000 trọng số duy nhất cho cấu hình $N = 8$ và thiết bị chỉ có 360 bộ nhân (XCZU3EG), tiết kiệm tài nguyên DSP này là rất quan trọng.

Lưu ý rằng, mặc dù kiến trúc CNN được làm cho khá tổng quát, quá trình từ phát triển mô hình đến huấn luyện đến triển khai FPGA có phần được điều chỉnh cho số kênh nhất định, do đó chỉ có thử nghiệm $N = 8$ được thực hiện và hoàn thành. Đối với hệ thống cụ thể nơi số kênh không chắc chắn hoặc thay đổi, đây có thể là lý do để sử dụng giải pháp QRD-RLS dạng đóng để mở rộng hơn cho số kênh khác nhau.

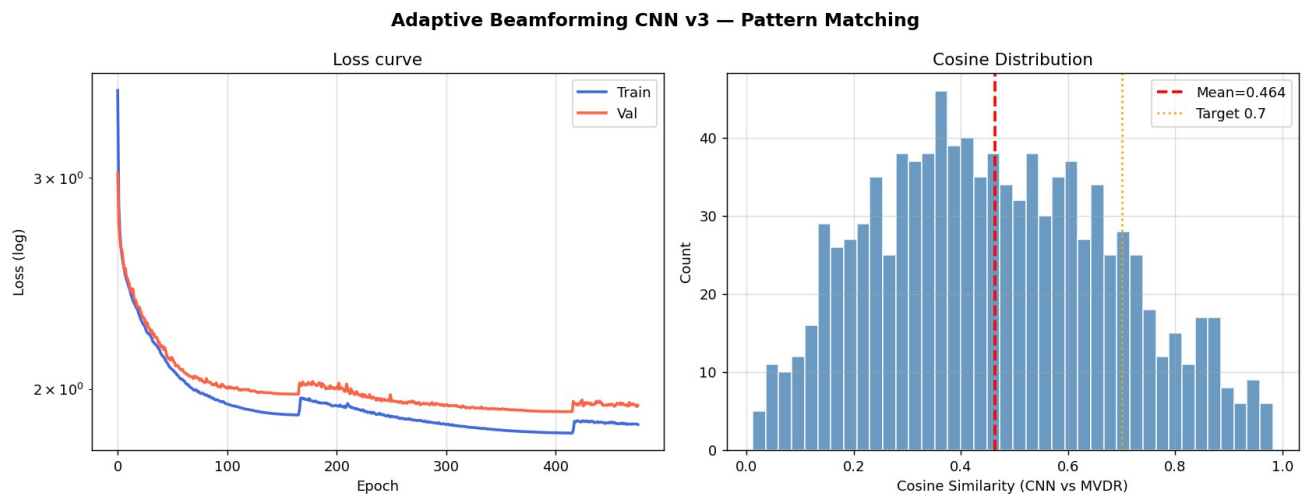
Chúng ta có thể đạt được tiết kiệm logic/DSP thậm chí nhiều hơn bằng cách "gập" hoặc sử dụng lại cùng các bộ nhân cho nhiều lớp. Đối với các mạng nơ-ron lớn, phức tạp với hàng trăm nghìn tham số trọng số trở lên, có thể xem xét các kiến trúc tăng tốc phần cứng khác như Đơn vị Xử lý Tensor (TPU) của Google sử dụng mảng systolic 256×256 rộng để thực hiện 65.536 phép nhân-tích lũy (MAC) mỗi chu kỳ.



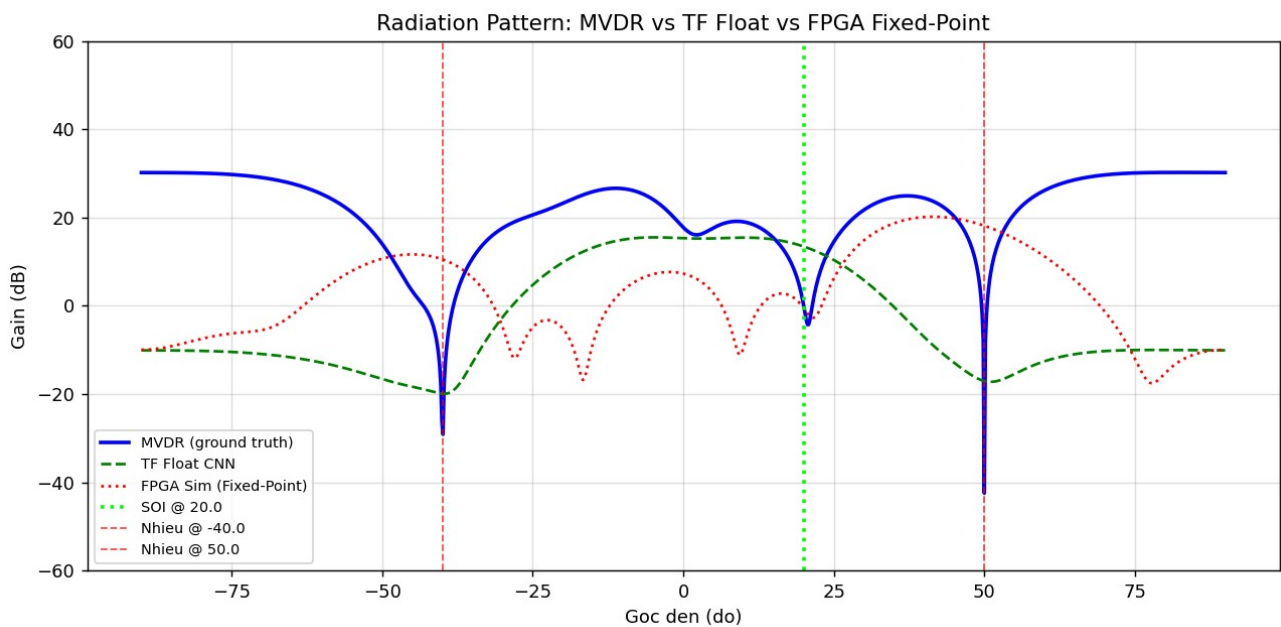
[Hình 4.5: Đơn vị Xử lý Tensor (TPU) Google – Bộ nhân ma trận]

Chương 5: Các kết quả đạt được & hướng phát triển

Kết quả huấn luyện bộ trọng số:



[Hình 5.1: Kết quả huấn luyện hàm Loss và phân phối độ tương đồng Cosine của mô hình Adaptive Beamforming CNN]



[Hình 5.2: Đồ thị bức xạ của mô hình sau khi huấn luyện so với lý tưởng]

5.2 Kết luận và hướng phát triển:

Sau một thời gian tập trung nghiên cứu, đề tài “Nghiên cứu và triển khai kỹ thuật Beamforming sử dụng mạng neuron trên FPGA” đã hoàn thành các mục tiêu cơ bản đặt ra về mặt lý thuyết và mô phỏng thuật toán trên phần mềm. Nghiên cứu đã làm chủ được cơ chế định hình búp sóng thích nghi MVDR cho mảng anten 8 phần tử, đồng thời đề xuất thành công cấu trúc mạng CNN chuyên dụng tối giản giúp tách biệt kênh thực và kênh ảo qua các bộ lọc kích thước 5×4 . Kết quả này bước đầu chứng minh tính khả thi của việc ứng dụng mạng học sâu vào bài toán xử lý mảng anten và đặt nền móng giải thuật vững chắc cho các thiết kế vi mạch trong tương lai.

Về ưu điểm, đề tài đã cải tiến thành công hàm Loss truyền thống bằng cách tích hợp thêm thành phần Cosine Similarity, giúp mạng CNN tối ưu hóa chính xác hướng của vector trọng số phức để hướng búp sóng về phía tín hiệu mong muốn. Bên cạnh đó, mô hình được thiết kế theo hướng tối ưu hóa tài nguyên phần cứng ngay từ đầu, giảm thiểu kích thước không gian phẳng xuống còn 50 đặc trưng và kiểm chứng thành công khả năng lượng tử hóa Post-Training Quantization khi ép toàn bộ trọng số về dải số nguyên 8-bit có dấu (Signed INT8) từ -128 đến 127 một cách gọn gàng, đáp ứng tốt ràng buộc lưu trữ trên các thanh ghi phần cứng.

Về nhược điểm, do giới hạn về mặt thời gian nên đề tài hiện tại mới dừng lại ở bước huấn luyện, mô phỏng thuật toán trên phần mềm bằng các tập dữ liệu giả lập toán học và chưa hoàn thiện quy trình viết mã mô tả phần cứng RTL cũng như thực nghiệm nạp mạch lên Kit phát triển FPGA thực tế. Ngoài ra, việc gò ép cấu trúc tích chập xuống còn một bộ lọc duy nhất nhằm định hướng tiết kiệm tài nguyên tương lai đã phần nào làm giảm năng lực biểu diễn toán học của mạng CNN khi đối mặt với kích bản nhiều nguồn nhiễu đồng thời, kết hợp với việc lượng tử hóa thô bạo ở chế độ Full INT8 trên phần mềm làm nảy sinh sai số làm tròn pha, gây ảnh hưởng đến tỷ số SINR trên đồ thị bức xạ mô phỏng.

Về hướng phát triển, trong thời gian tới đề tài sẽ tập trung áp dụng kỹ thuật huấn luyện nhận biết lượng tử hóa (Quantization-Aware Training - QAT) trực tiếp trên Python để chủ động bù trừ sai số pha, đồng thời tiến hành xây dựng mã nguồn mô tả phần cứng RTL bằng SystemVerilog theo cấu hình lượng tử hóa Mixed-Precision 16×8 nhằm tận dụng tối đa khối tính toán DSP Slices (DSP48E1) trên các dòng chip FPGA hiện đại để tăng độ mịn của pha. Đồng thời, nghiên cứu sẽ nói lỏng kiến trúc mạng lên nhiều bộ lọc tích chập hơn, thiết kế các mạch tính toán song song song (Pipeline) và tiến tới nạp thử nghiệm thực tế Over-The-Air (OTA) trên các kit phát triển FPGA như Xilinx Artix-7 hay Kintex-7 kết hợp mạch RF Front-End để đánh giá toàn diện hiệu năng hệ thống trong môi trường thời gian thực.

Phụ lục A: Mã nguồn phần mềm

```
r"""
=====

Adaptive Beamforming CNN
=====

"""

import numpy as np, os, pathlib

os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks
from scipy.linalg import solve
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

print(f"TensorFlow {tf.__version__} | GPU: {len(tf.config.list_physical_devices('GPU'))>0}")

# — Cấu hình —————
BASE_DIR = pathlib.Path(r'D:\Kien\DA1\thuc_hien')

class Config:
    N          = 8
    d          = 0.5
    K          = 128
    MAX_INTER  = 7
    K_H        = 5          # Kernel height
```

```

K_W      = 4          # Kernel width

FLAT_DIM  = 5 * 5 * 2  # = 50

DENSE_DIM = 4 * 8      # = 32 (= 4N)

TRAIN_SIZE = 20000    # 10000 → 20000

VAL_SIZE   = 3000

TEST_SIZE  = 1000

BATCH_SIZE = 32

EPOCHS     = 500

LR          = 1e-3

SEED        = 42

MODEL_FILE = str(BASE_DIR / 'best_tf_model.keras')

PLOT_FILE  = str(BASE_DIR / 'beamforming_tf_results.png')

cfg = Config()

tf.random.set_seed(cfg.SEED)

np.random.seed(cfg.SEED)

# — Data Generation —————

def steering_vector(N, sin_theta, d=0.5):

    return np.exp(1j * 2 * np.pi * d * sin_theta * np.arange(N))

def generate_scenario(cfg):

    N, K, d = cfg.N, cfg.K, cfg.d

    soi_angle = np.random.uniform(-60, 60)

    soi_sin    = np.sin(np.deg2rad(soi_angle))

    soi_power  = 10 ** (np.random.uniform(10, 30) / 10)

    sv_soi     = steering_vector(N, soi_sin, d)

    t          = np.arange(K)

    soi_sig    = (np.sqrt(soi_power) *

```

```

        np.exp(1j*2*np.pi*np.random.uniform(0.1, 0.4)*t) *
        (np.random.randn(K) + 1j*np.random.randn(K)) / np.sqrt(2))
X = np.outer(sv_soi, soi_sig)
for _ in range(np.random.randint(1, cfg.MAX_INTER + 1)):
    ang = np.random.uniform(-80, 80)
    while abs(ang - soi_angle) < 10:
        ang = np.random.uniform(-80, 80)
    ip = 10 ** (np.random.uniform(20, 40) / 10)
    svi = steering_vector(N, np.sin(np.deg2rad(ang)), d)
    si = (np.sqrt(ip) *
        np.exp(1j*2*np.pi*np.random.uniform(0.1, 0.4)*t) *
        (np.random.randn(K) + 1j*np.random.randn(K)) / np.sqrt(2))
    X += np.outer(svi, si)
X += (np.random.randn(N, K) + 1j*np.random.randn(N, K)) / np.sqrt(2)
R = (X @ X.conj().T) / K
Ria = solve(R, sv_soi, assume_a='gen')
w = Ria / (sv_soi.conj() @ Ria + 1e-12)
return R, sv_soi, w

def make_sample(cfg):
    N = cfg.N
    R, sv, w = generate_scenario(cfg)

    w_norm = w / (np.linalg.norm(w) + 1e-12)

    sv_row = sv.reshape(1, N)
    mat = np.vstack([R, sv_row])
    mat /= (np.max(np.abs(mat)) + 1e-12)
    tensor = np.stack([mat.real, mat.imag], axis=-1).astype(np.float32)
    label = np.concatenate([w_norm.real, w_norm.imag]).astype(np.float32)

```



```

    return tensor, label

def build_dataset(cfg, n, tag=''):
    X = np.zeros((n, cfg.N+1, cfg.N, 2), dtype=np.float32)
    Y = np.zeros((n, 2*cfg.N), dtype=np.float32)
    for i in range(n):
        X[i], Y[i] = make_sample(cfg)
        if (i+1) % 5000 == 0:
            print(f" {tag}{i+1}/{n}")
    return X, Y

# — Custom Layer (thay Lambda, tránh lỗi load_model) —
@tf.keras.utils.register_keras_serializable(package='BeamformingCNN')
class ChannelSlice(keras.layers.Layer):
    def __init__(self, ch_start: int, ch_end: int, **kwargs):
        super().__init__(**kwargs)
        self.ch_start = ch_start
        self.ch_end = ch_end
    def call(self, x):
        return x[:, :, :, self.ch_start:self.ch_end]
    def get_config(self):
        c = super().get_config()
        c.update({'ch_start': self.ch_start, 'ch_end': self.ch_end})
        return c

# — Kiến trúc CNN (giữ khớp FPGA) —
def build_cnn(cfg) -> keras.Model:
    N = cfg.N
    inp = keras.Input(shape=(N+1, N, 2), name='covariance_input')
    real_ch = ChannelSlice(0, 1, name='split_real')(inp)

```

```

imag_ch  = ChannelSlice(1, 2, name='split_imag')(inp)
conv_real = layers.Conv2D(1, (cfg.K_H, cfg.K_W), padding='valid',
                           activation='relu', name='conv_real_ch')(real_ch)
conv_imag = layers.Conv2D(1, (cfg.K_H, cfg.K_W), padding='valid',
                           activation='relu', name='conv_imag_ch')(imag_ch)
merged    = layers.Concatenate(axis=-1, name='concat_channels')([conv_real, conv_imag])
flat      = layers.Flatten(name='flatten')(merged)
x         = layers.Dense(cfg.DENSE_DIM, activation='relu',
                           name='dense_hidden')(flat)
out       = layers.Dense(2*N, activation='linear',
                           name='beamforming_weights')(x)

return keras.Model(inputs=inp, outputs=out, name='AdaptiveBeamformingCNN')

```

— Loss function: Cosine + Unit-Norm Pattern dB —————

```
def combined_loss(y_true, y_pred):
```

— 1. Cosine loss (scale-invariant) —————

```

N      = tf.shape(y_true)[1] // 2
wr_t   = y_true[:, :N]; wi_t = y_true[:, N:]
wr_p   = y_pred[:, :N]; wi_p = y_pred[:, N:]

dot_r   = tf.reduce_sum(wr_t*wr_p + wi_t*wi_p, axis=1)
dot_i   = tf.reduce_sum(wr_t*wi_p - wi_t*wr_p, axis=1)
dot_abs = tf.sqrt(tf.square(dot_r) + tf.square(dot_i) + 1e-12)
n_t     = tf.sqrt(tf.reduce_sum(tf.square(y_true), axis=1, keepdims=True) + 1e-12)
n_p     = tf.sqrt(tf.reduce_sum(tf.square(y_pred), axis=1, keepdims=True) + 1e-12)
cos_loss = 1.0 - tf.reduce_mean(dot_abs / (tf.squeeze(n_t) * tf.squeeze(n_p)))

```

— 2. Unit-norm pattern loss (không normalize by max) —————

```

wr_t_n = wr_t / n_t; wi_t_n = wi_t / n_t # unit-norm true
wr_p_n = wr_p / n_p; wi_p_n = wi_p / n_p # unit-norm pred

# Steering matrix:  $A[\theta, n] = \exp(j 2\pi 0.5 \sin(\theta) n)$ , 361 điểm
sin_t = tf.linspace(-1.0, 1.0, 361)
n_range = tf.cast(tf.range(N), tf.float32)
phase = 2.0 * np.pi * 0.5 * tf.tensordot(sin_t, n_range, axes=0) # (361, N)
A_r = tf.cos(phase)
A_i = tf.sin(phase)

def power_pattern(wr, wi):
    """ $G = |w^H a(\theta)|^2$ , shape (batch, 361)"""
    Gr = tf.matmul(wr, A_r, transpose_b=True) + tf.matmul(wi, A_i, transpose_b=True)
    Gi = tf.matmul(wr, A_i, transpose_b=True) - tf.matmul(wi, A_r, transpose_b=True)
    return tf.square(Gr) + tf.square(Gi) + 1e-12

G_t = power_pattern(wr_t_n, wi_t_n) # (batch, 361), unit-norm true
G_p = power_pattern(wr_p_n, wi_p_n) # (batch, 361), unit-norm pred

log10 = tf.math.log(10.0)
db_t = tf.clip_by_value(10.0 * tf.math.log(G_t) / log10, -60.0, 12.0)
db_p = tf.clip_by_value(10.0 * tf.math.log(G_p) / log10, -60.0, 12.0)

w_null = 1.0 + 9.0 * tf.cast(db_t < -20.0, tf.float32)
pat_mse = tf.reduce_mean(w_null * tf.square(db_p - db_t))

alpha = 0.005

return cos_loss + alpha * pat_mse

```

```

# — Huấn luyện —————
def train(cfg):
    BASE_DIR.mkdir(parents=True, exist_ok=True)

    print("► Sinh dữ liệu huấn luyện...")

    X_tr, Y_tr = build_dataset(cfg, cfg.TRAIN_SIZE, 'Train ')
    X_vl, Y_vl = build_dataset(cfg, cfg.VAL_SIZE, 'Val ')
    X_te, Y_te = build_dataset(cfg, cfg.TEST_SIZE, 'Test ')

    model = build_cnn(cfg)
    model.summary()

    # LR schedule: warmup 5 epoch, sau đó cosine decay đến 1e-6
    total_steps = (cfg.TRAIN_SIZE // cfg.BATCH_SIZE) * cfg.EPOCHS
    warmup_steps = (cfg.TRAIN_SIZE // cfg.BATCH_SIZE) * 5

    lr_schedule = keras.optimizers.schedules.CosineDecayRestarts(
        initial_learning_rate = cfg.LR,
        first_decay_steps     = total_steps // 3,
        t_mul=1.5, m_mul=0.8, alpha=1e-6
    )

    model.compile(
        optimizer = keras.optimizers.Adam(lr_schedule),
        loss      = combined_loss,
        metrics   = ['mae']
    )

    cbs = [
        callbacks.EarlyStopping(monitor='val_loss', patience=60,

```

```

        restore_best_weights=True, verbose=1),
        callbacks.ModelCheckpoint(cfg.MODEL_FILE, monitor='val_loss',
                                   save_best_only=True, verbose=0),
        callbacks.TerminateOnNaN(),
    ]

print("► Bắt đầu huấn luyện...")

hist = model.fit(
    X_tr, Y_tr,
    validation_data = (X_vl, Y_vl),
    batch_size      = cfg.BATCH_SIZE,
    epochs          = cfg.EPOCHS,
    callbacks       = cbs,
    verbose         = 1
)

# — Đánh giá —————
print("\n► Đánh giá trên Test set...")

Y_pred = model.predict(X_te, verbose=0)

N = cfg.N

w_cnn  = Y_pred[:, :N] + 1j * Y_pred[:, N:]
w_mvdr = Y_te[:, :N] + 1j * Y_te[:, N:]

cos_sim = np.array([
    np.abs(w_cnn[i].conj() @ w_mvdr[i]) /
    (np.linalg.norm(w_cnn[i]) * np.linalg.norm(w_mvdr[i]) + 1e-12)
    for i in range(len(w_cnn))
])

print(f" Cosine similarity : {np.mean(cos_sim):.4f} ± {np.std(cos_sim):.4f}")

print(f" > 0.90           : {np.sum(cos_sim>0.90)}/{len(cos_sim)}
({100*np.mean(cos_sim>0.90):.1f}%)")

```

```

print(f" > 0.70 : {np.sum(cos_sim>0.70)}/{len(cos_sim)}
({100*np.mean(cos_sim>0.70):.1f}%")

# — Plot —————
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle('Adaptive Beamforming CNN v3 – Pattern Matching',
             fontsize=13, fontweight='bold')

ax = axes[0]
ax.semilogy(hist.history['loss'], label='Train', color='royalblue', lw=2)
ax.semilogy(hist.history['val_loss'], label='Val', color='tomato', lw=2)
ax.set_xlabel('Epoch'); ax.set_ylabel('Loss (log)')
ax.set_title('Loss curve'); ax.legend(); ax.grid(alpha=0.4)

ax = axes[1]
ax.hist(cos_sim, bins=40, color='steelblue', edgecolor='white', alpha=0.8)
ax.axvline(np.mean(cos_sim), color='r', ls='--', lw=2,
           label=f'Mean={np.mean(cos_sim):.3f}')
ax.axvline(0.7, color='orange', ls=':', lw=1.5, label='Target 0.7')
ax.set_xlabel('Cosine Similarity (CNN vs MVDR)')
ax.set_ylabel('Count'); ax.set_title('Cosine Distribution')
ax.legend(); ax.grid(alpha=0.4)

plt.tight_layout()
plt.savefig(cfg.PLOT_FILE, dpi=130, bbox_inches='tight')
print(f" Plot: {cfg.PLOT_FILE}")

# — TFLite —————
print("\n► TFLite quantization (16×8 mode)...")
def representative_dataset():

```

```

    for i in range(0, min(5000, cfg.TRAIN_SIZE), 5):
        yield [X_tr[i:i+1]]

conv = tf.lite.TFLiteConverter.from_keras_model(model)
conv.optimizations = [tf.lite.Optimize.DEFAULT]
conv.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
conv.inference_input_type = tf.float32
conv.inference_output_type = tf.float32
conv.representative_dataset = representative_dataset
try:
    tflm = conv.convert()
    p = str(BASE_DIR / 'beamforming_quantized.tflite')
    open(p, 'wb').write(tflm)
    print(f" TFLite: {len(tflm)/1024:.1f} KB → {p}")
except Exception as e:
    print(f" (Quantization skipped: {e})")

print(f"\n✓ Model: {cfg.MODEL_FILE}")
return model, hist

if __name__ == '__main__':
    model, hist = train(cfg)

```

Phụ lục B: Mã nguồn thiết kế SystemVerilog

TÀI LIỆU THAM KHẢO

- [1] "Adaptive beamforming for radar: Floating-point QRD+WBS in an FPGA," Xilinx, Tech. Rep. WP452, 2014.
- [2] "Application note 506: QR matrix decomposition," Altera, Tech. Rep. 2.0, 2008.
- [3] "Massive MIMO and beamforming: The signal processing behind the 5G buzzwords," Analog Devices, Tech. Rep., 2017.
- [4] "Phased array antenna patterns – part 1: Linear array beam characteristics and array factor," Analog Devices, Tech. Rep., 2020.
- [5] "Phased array antenna patterns – part 2: Grating lobes and beam squint," Analog Devices, Tech. Rep., 2020.
- [6] "Verizon completes first successful end-to-end fully virtualized 5G data session in the world," 2020.
- [7] A. Hirose, Complex-Valued Neural Networks, 2nd ed. Springer, 2012.
- [8] An in-depth look at Google's first Tensor Processing Unit (TPU), Google Cloud Blog, 2017.
- [9] B. D. V. Veen và K. M. Buckley, "Beamforming: A versatile approach to spatial filtering," IEEE ASSP Magazine, 1998.
- [10] B. Luitjen et al., "Deep learning for fast adaptive beamforming," ICASSP, 2019.
- [11] C. Dick et al., "Real-time QRD-based beamforming on an FPGA platform," Asilomar, 2006.
- [12] C. R. Ward, P. J. Hargrave, và J. G. McWhirter, "A novel algorithm and architecture for adaptive digital beamforming," IEEE Trans. Antennas Propag., 1986.
- [13] C. Rader, "VLSI systolic arrays for adaptive nulling," IEEE Signal Processing Magazine, 1996.
- [14] D. A. Teman, Lecture series on hardware for deep learning part 5: The DL acceleration landscape, 2020.
- [15] D. Li, Q. Yin, P. Mu, và W. Guo, "Robust MVDR beamforming using the DOA matrix decomposition," IEEE-ISAS, 2011.
- [16] E. P. Tsakalaki et al., "Deterministic beamforming for enhanced vertical sectorization," EUCAP, 2012.
- [17] I. Goodfellow, Y. Bengio, và A. Courville, Deep Learning. MIT Press, 2016.
- [18] I. J. Goodfellow et al., Generative adversarial networks, 2014.
- [19] J. Guerci, Space-Time Adaptive Processing for Radar, 2nd ed. Artech House, 2015.
- [20] M. A. Richards, J. A. Scheer, và W. A. Holm, Principles of Modern Radar. IET, 2010.
- [21] M. Karkooti và J. R. Cavallaro, "FPGA implementation of matrix inversion using QRD-RLS algorithm," Asilomar Conf., 2005.
- [22] Q. Gao, L. Crockett, và R. Stewart, "Coarse angle rotation mode CORDIC based single processing element QR-RLS processor," EUSIPCO, 2009.
- [23] R. L. Maistre, "Xilinx unveils radio platform for second wave of 5G," 2020.
- [24] R. Mailloux, Phased Array Antenna Handbook, 2nd ed. Artech House, 2005.
- [25] S. Haykin, Adaptive Filter Theory, 5th ed. Pearson, 2014.

- [26] S. Rangan, T. S. Rappaport, và E. Erkip, "Millimeter-wave cellular wireless networks: Potentials and challenges," Proc. IEEE, vol. 102, no. 3, pp. 366–385, 2014.
- [27] T. J. O'Shea, J. Corgan, và T. C. Clancy, Convolutional radio modulation recognition networks, 2016.
- [28] T. Lin và Y. Zhu, "Beamforming design for large-scale antenna arrays using deep learning," IEEE WCL, 2020.
- [29] T. Maksymyuk et al., "Deep learning based massive MIMO beamforming for 5G mobile network," IEEE-IDAACS, 2018.
- [30] TensorFlow Lite – Model optimization guide, https://www.tensorflow.org/model_optimization/guide.
- [31] TensorFlow, <https://www.tensorflow.org/learn>.
- [32] Y. Shi et al., Deep learning for RF signal classification in unknown and dynamic spectrum environments, 2019.
- [33] J. Gentile, "FPGA-Based Adaptive Digital Beamforming Using Machine Learning for MIMO Systems," , May 2021.